

# Sarathi-Serve

Taming Throughput-Latency Tradeoff in  
LLM Inference

Presenters: Chau Tran, Seokjoo Cho



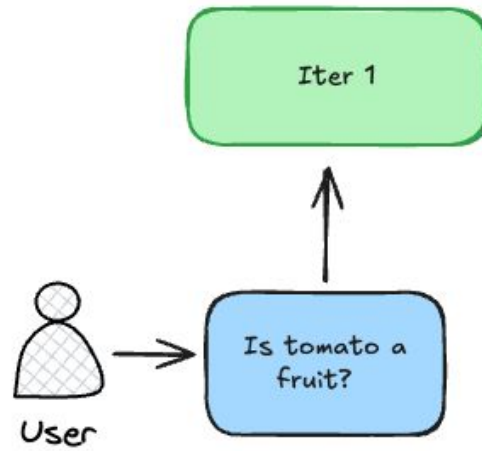
# Structure

1. **Latency-Throughput Trade-off:** Analyzing LLM batching policies
2. **Finding a free lunch:** Arithmetic Intensity Slack in LLM Inference
3. **Stall-free batching:** Leveraging chunked prefill to overcome the latency-throughput tradeoff
4. **Evaluations:** Key results and analysis

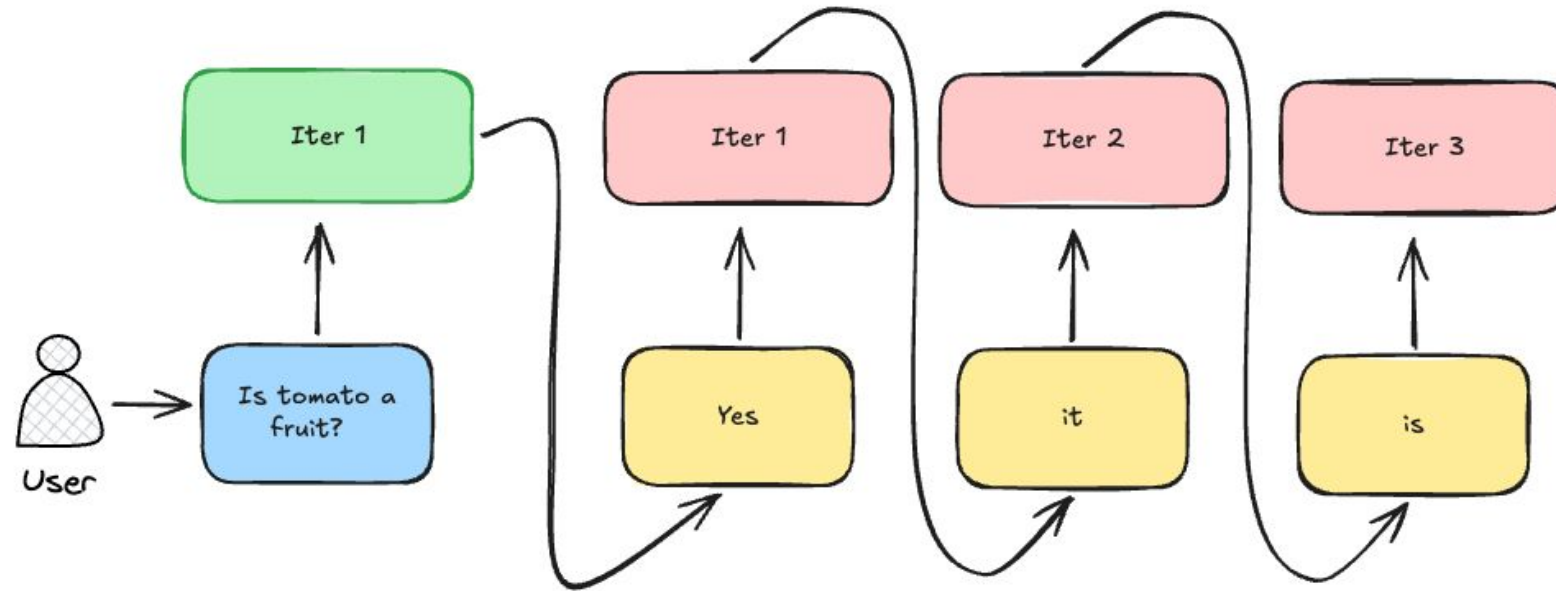
**What causes the latency-throughput tradeoff in LLM systems ?**



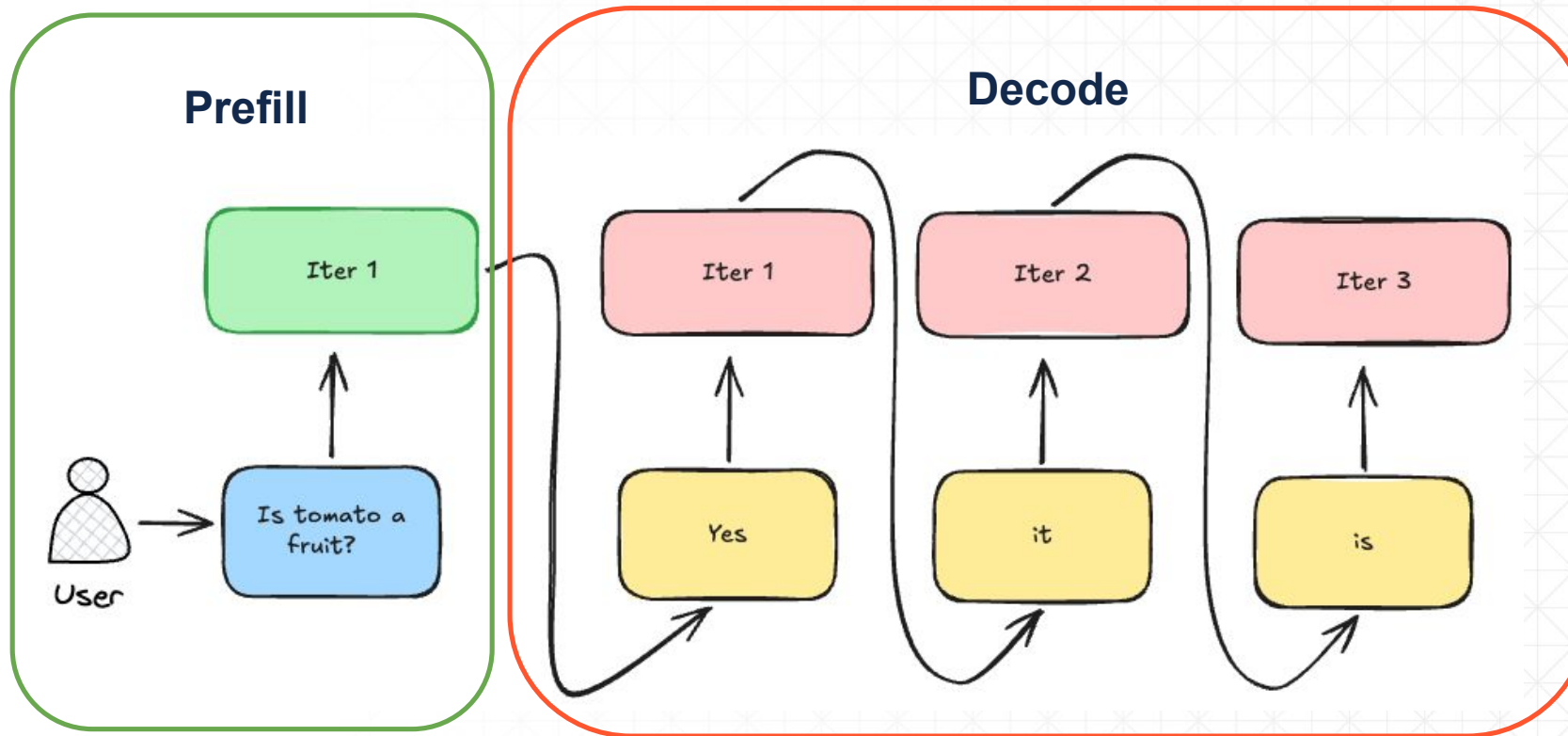
# LLM Inference 101



# LLM Inference 101



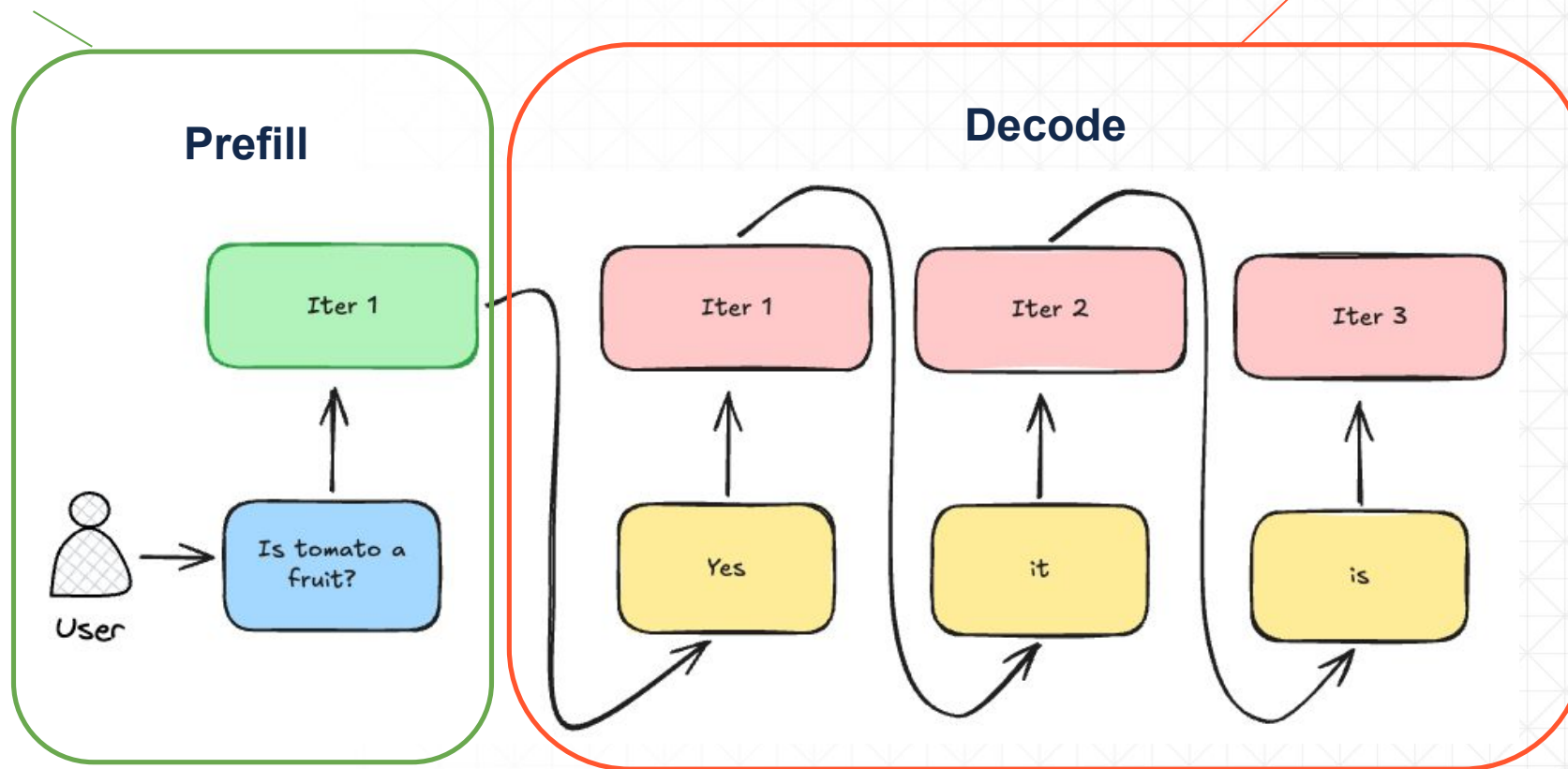
# LLM Inference 101



# LLM Inference 101

High GPU utilization  
High parallelism

Low GPU utilization  
Low parallelism





**How to improve parallelism during  
decoding phase?**



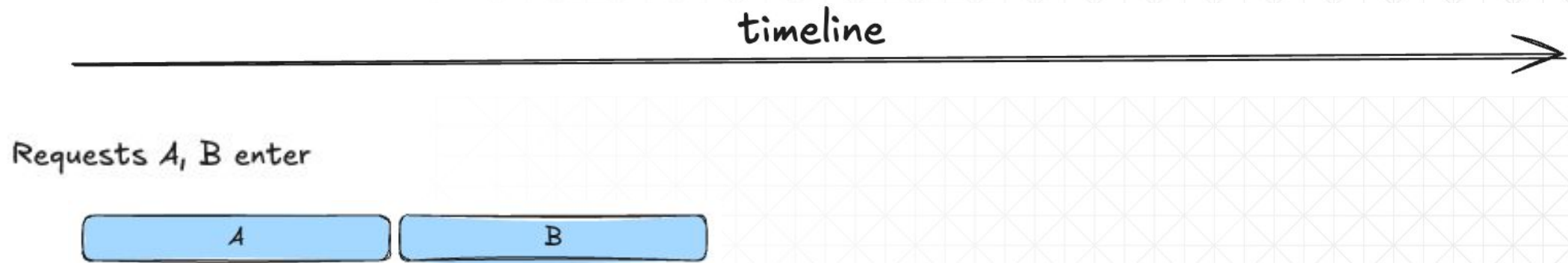
# How to improve parallelism during decoding phase?

**Batching**

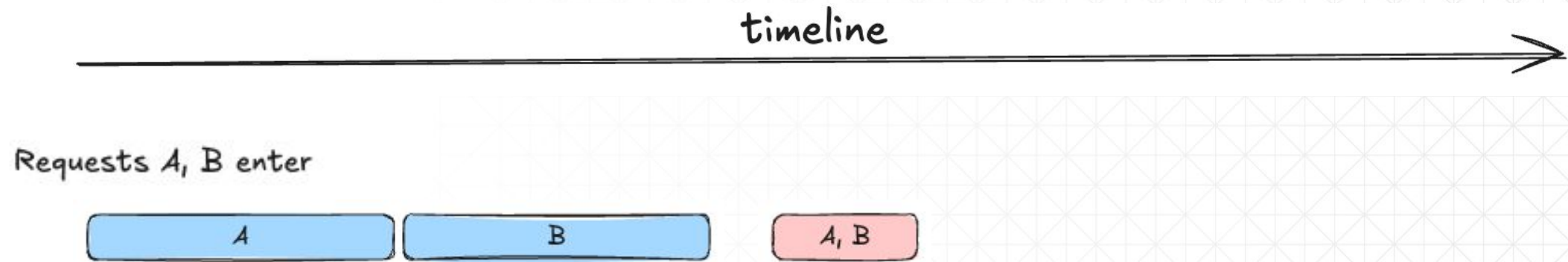
# Batching LLM Inference



# Batching LLM Inference

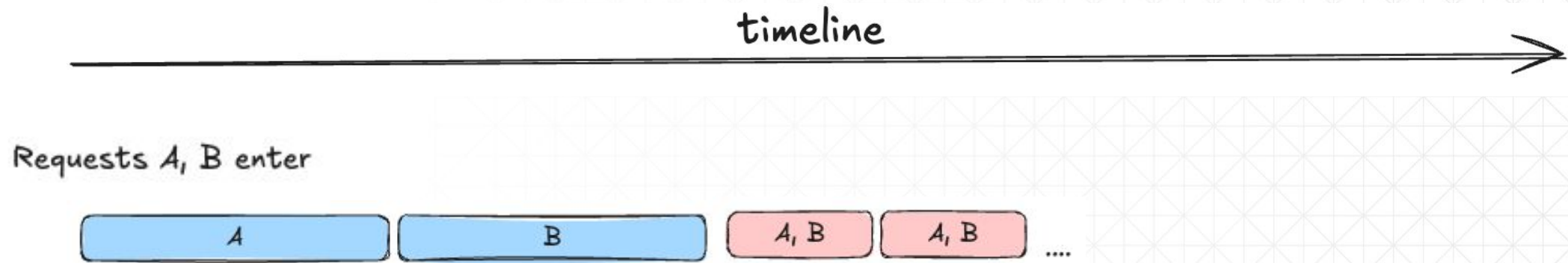


# Batching LLM Inference

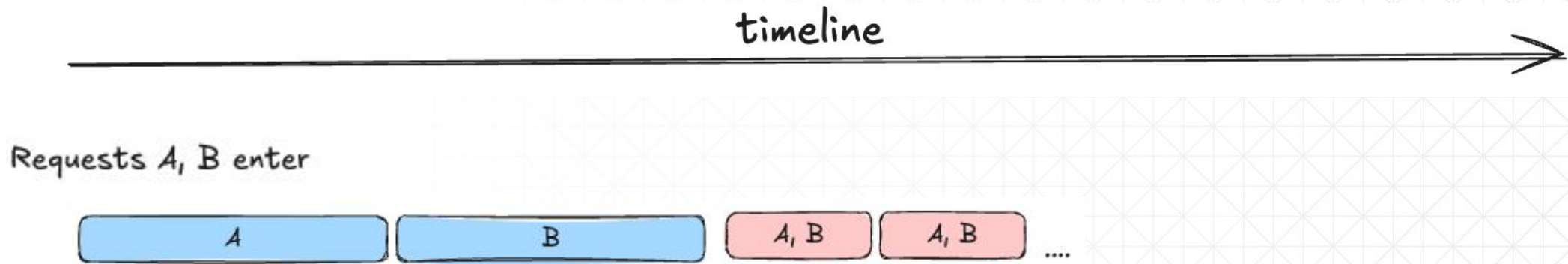




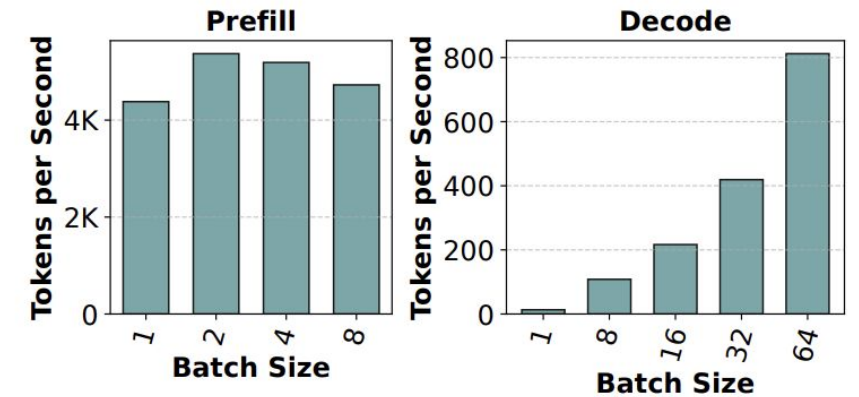
# Batching LLM Inference



# Batching LLM Inference



Decode efficiency increase **linearly** with batch size.  
**Higher batch size, higher throughput**



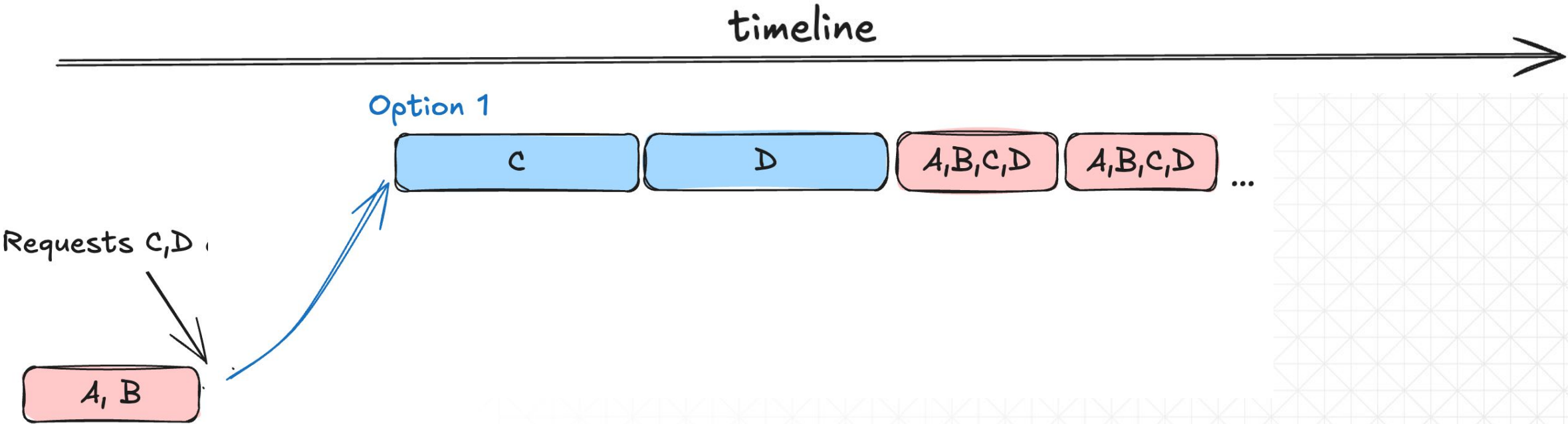
# Batching LLM Inference

timeline

Requests C,D enter

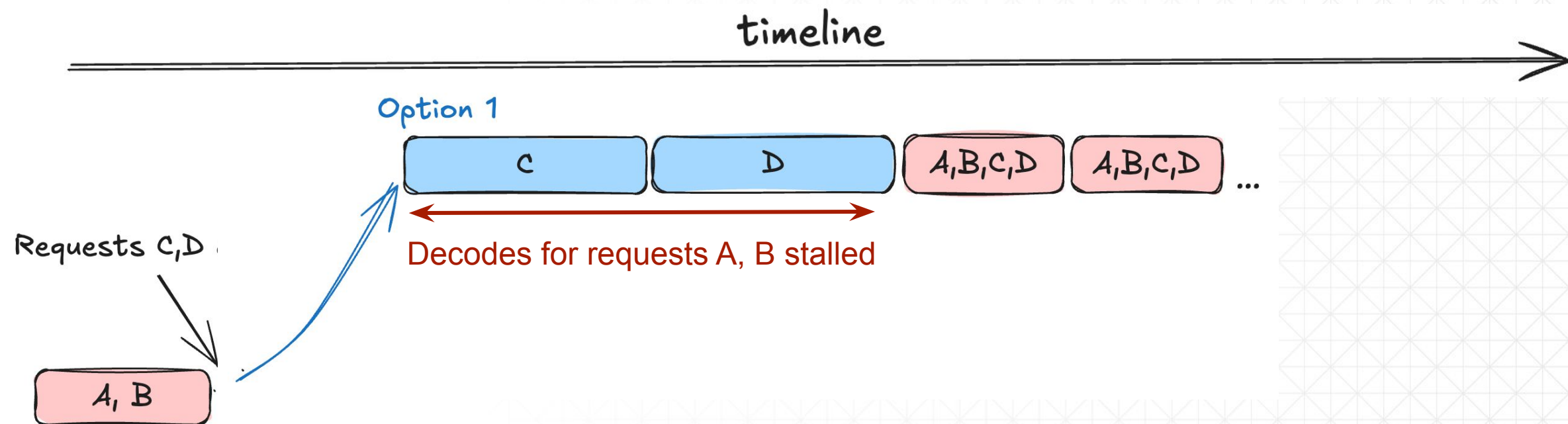
A, B

# Batching LLM Inference

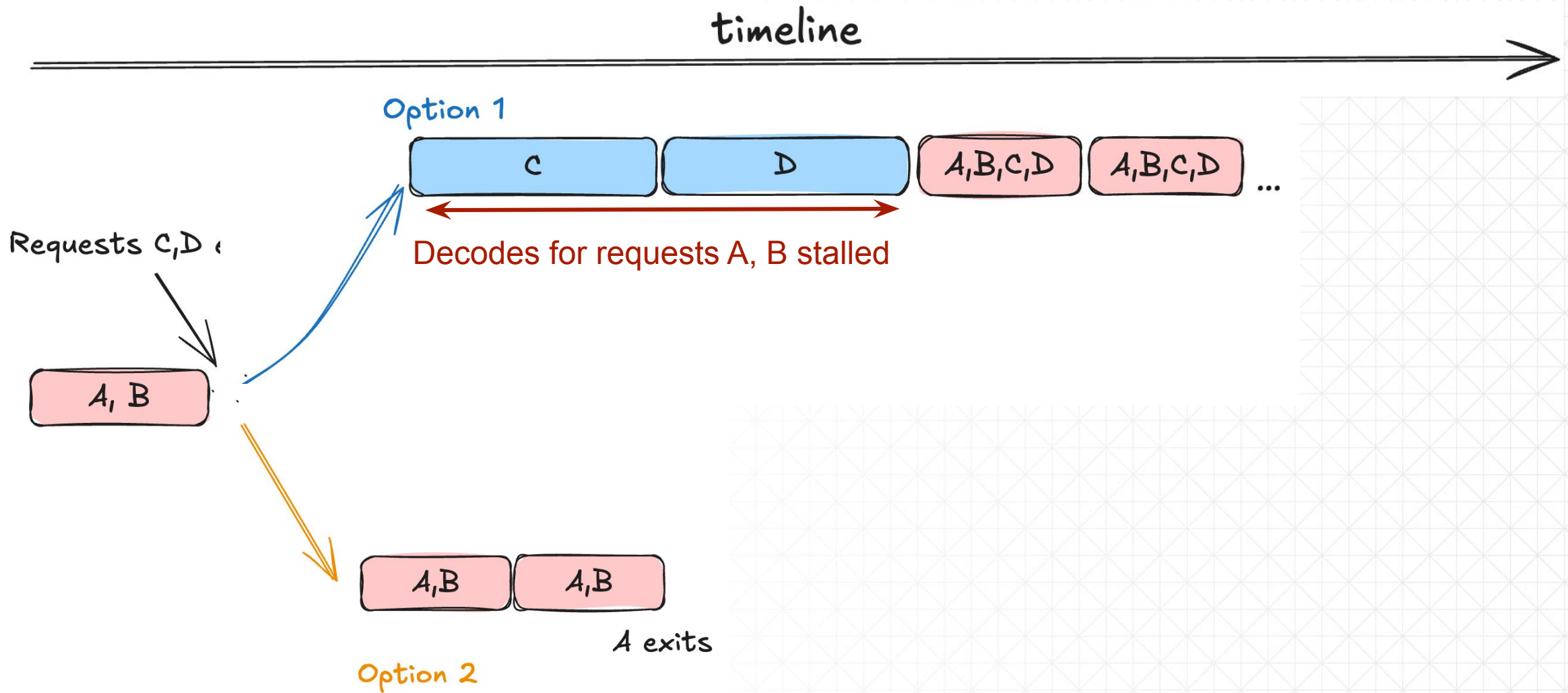




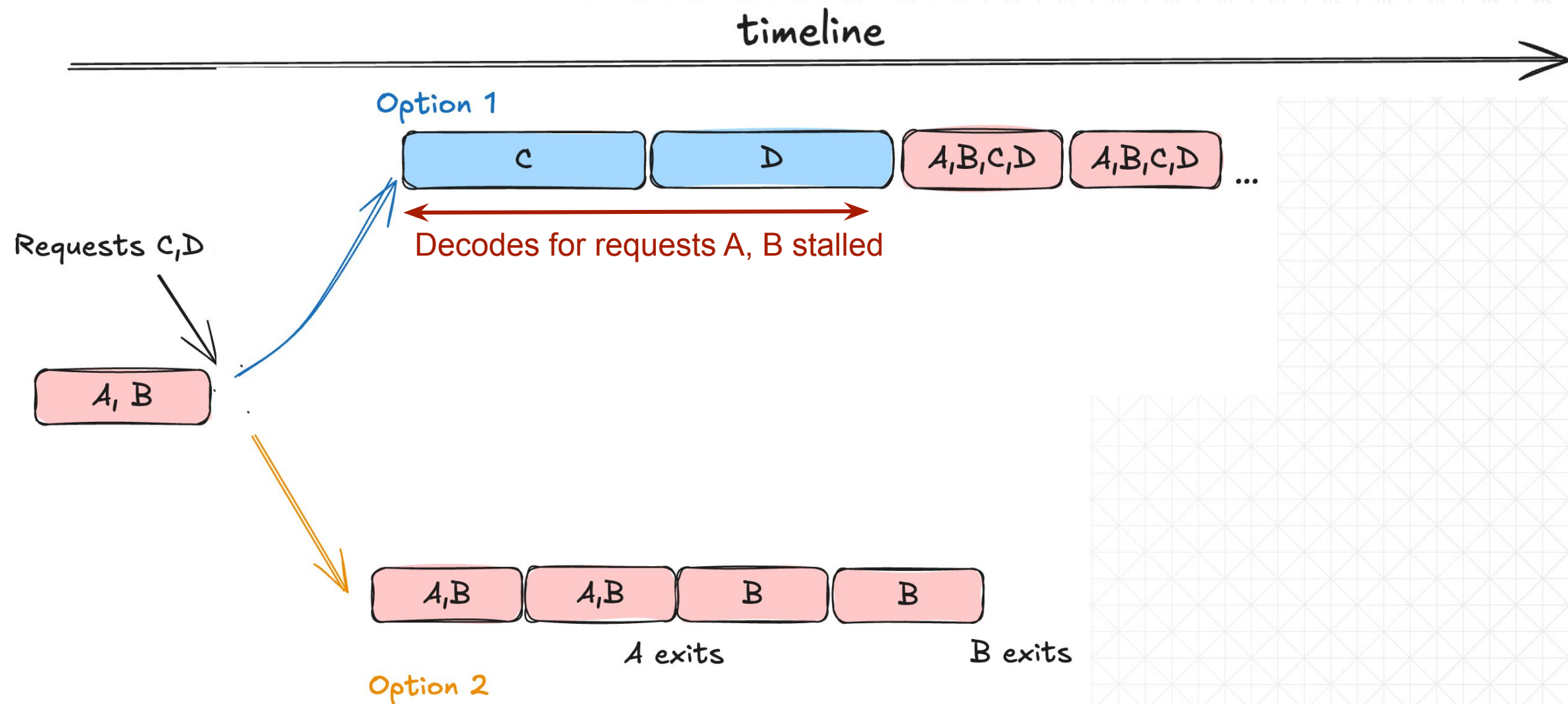
# Batching LLM Inference



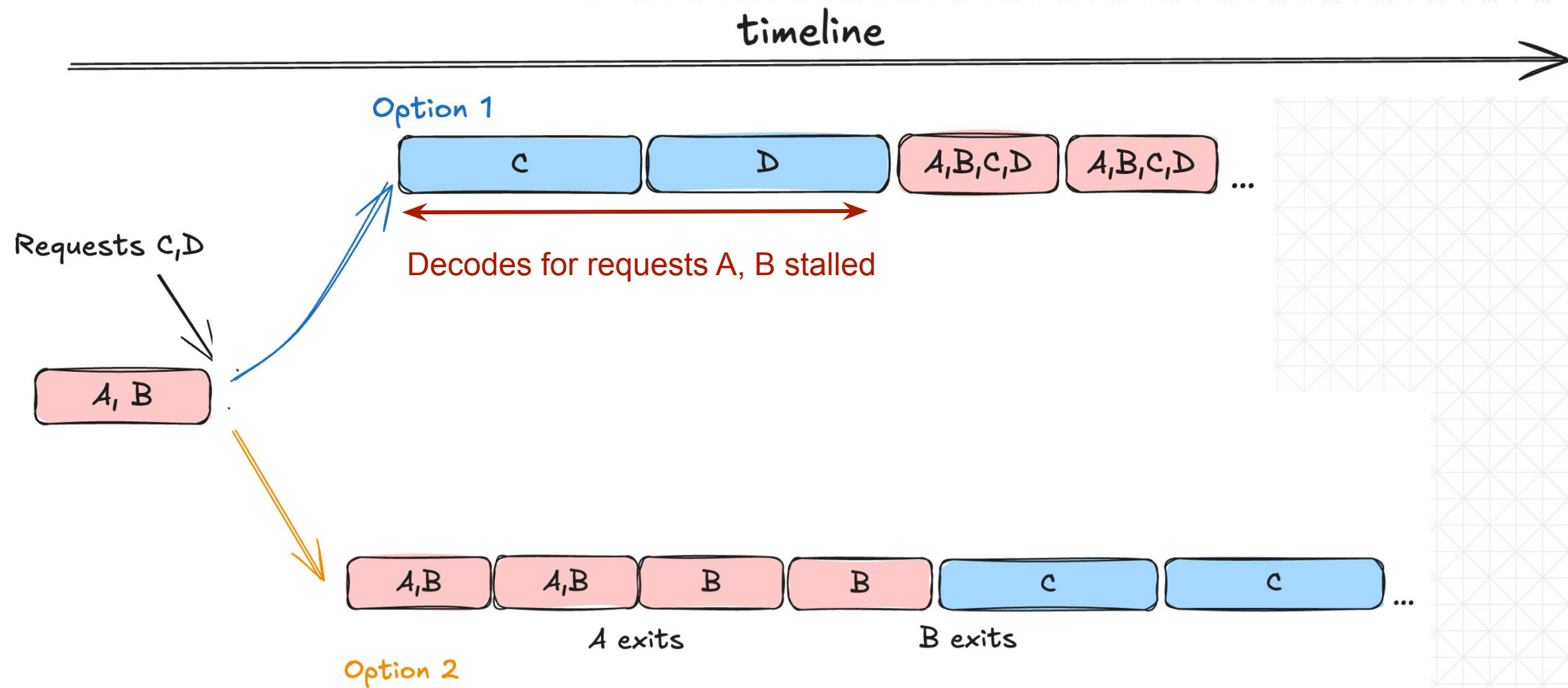
# Batching LLM Inference



# Batching LLM Inference

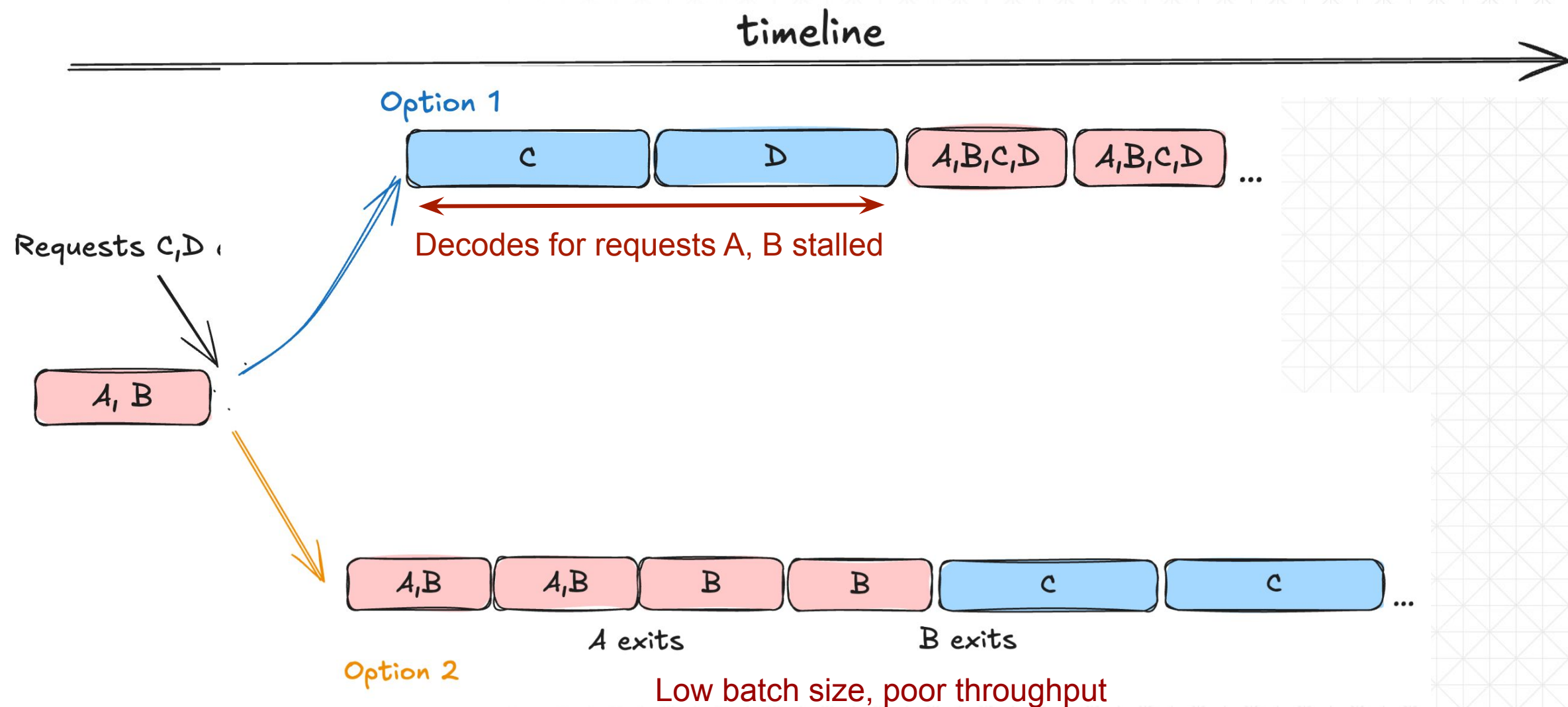


# Batching LLM Inference

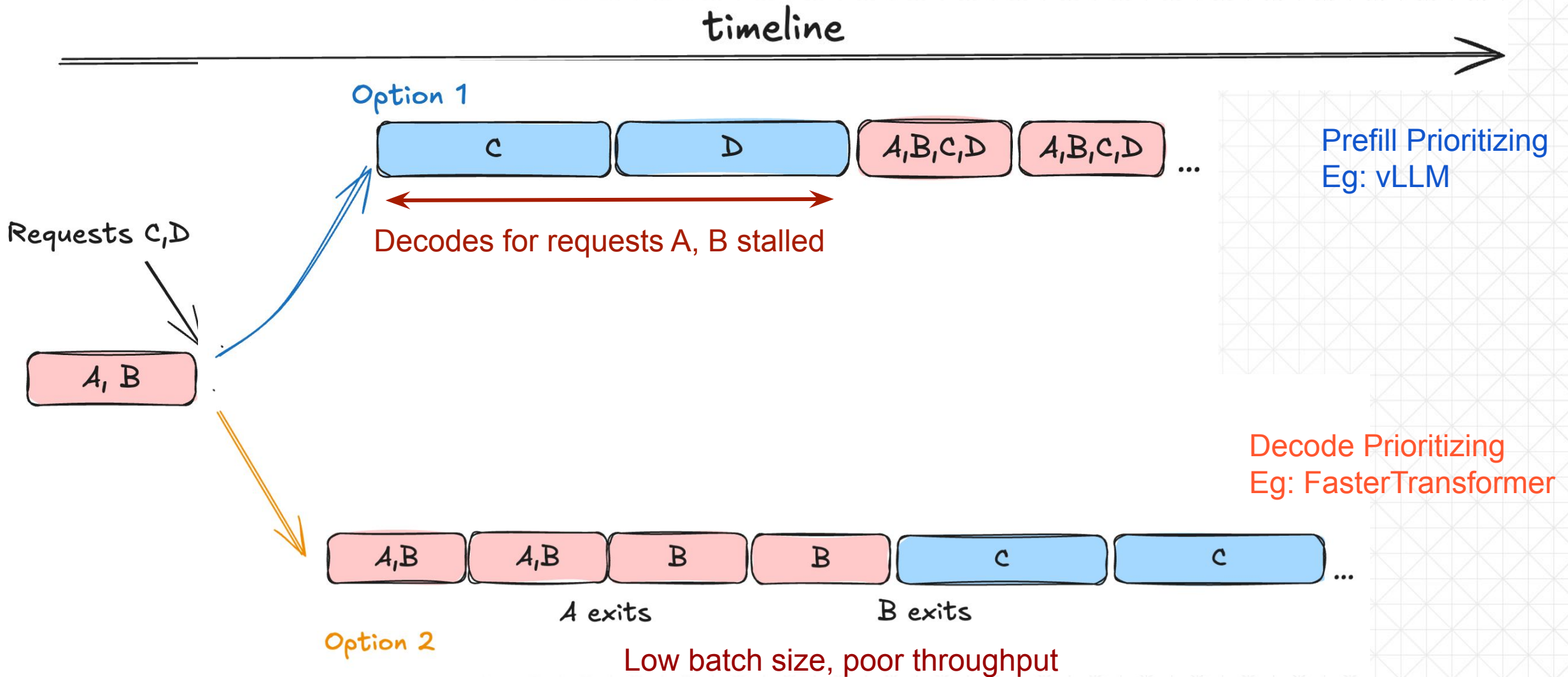




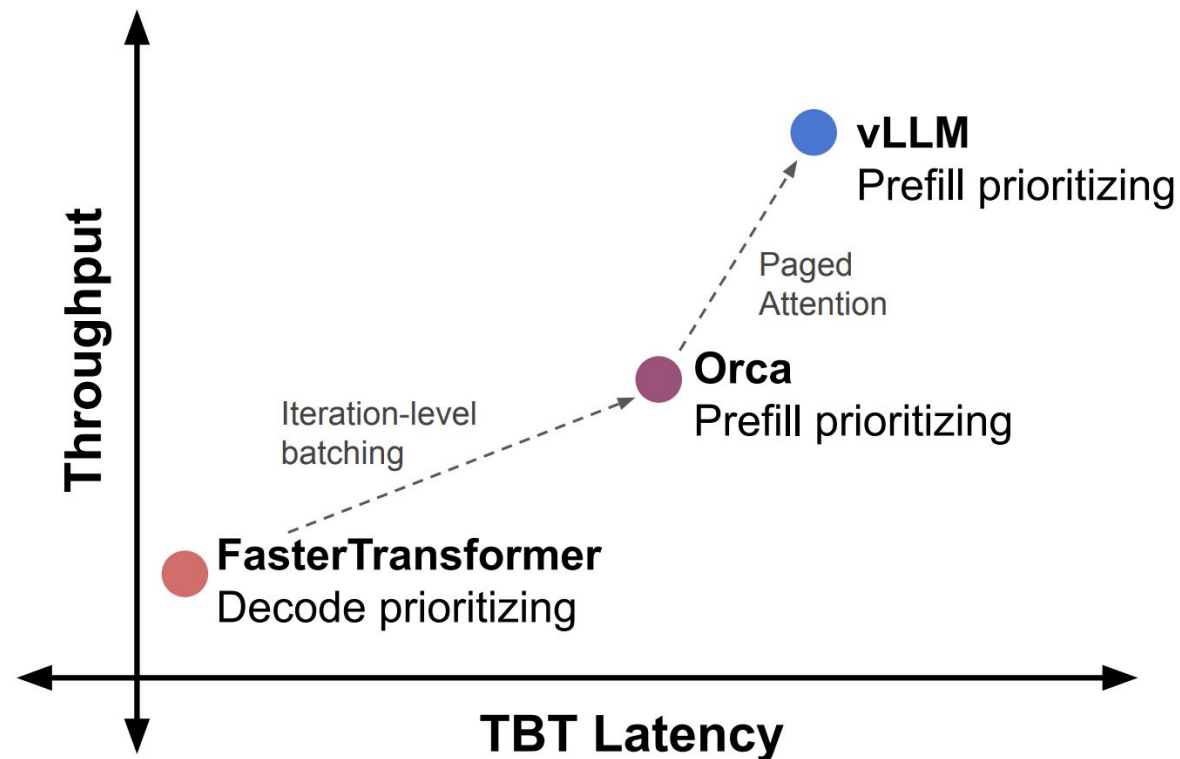
# Batching LLM Inference



# Batching LLM Inference

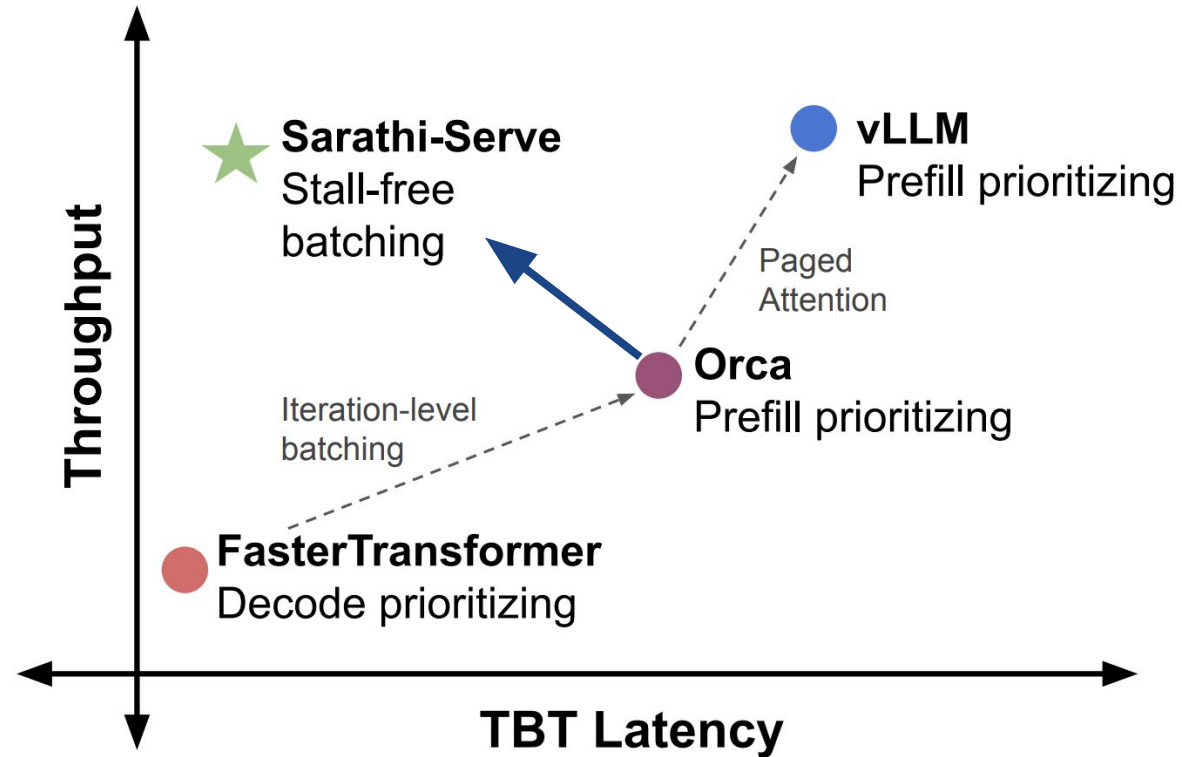


# The Latency-Throughput Tradeoff



Existing batching methods make a **harsh latency-throughput** tradeoff !

# The Latency-Throughput Tradeoff

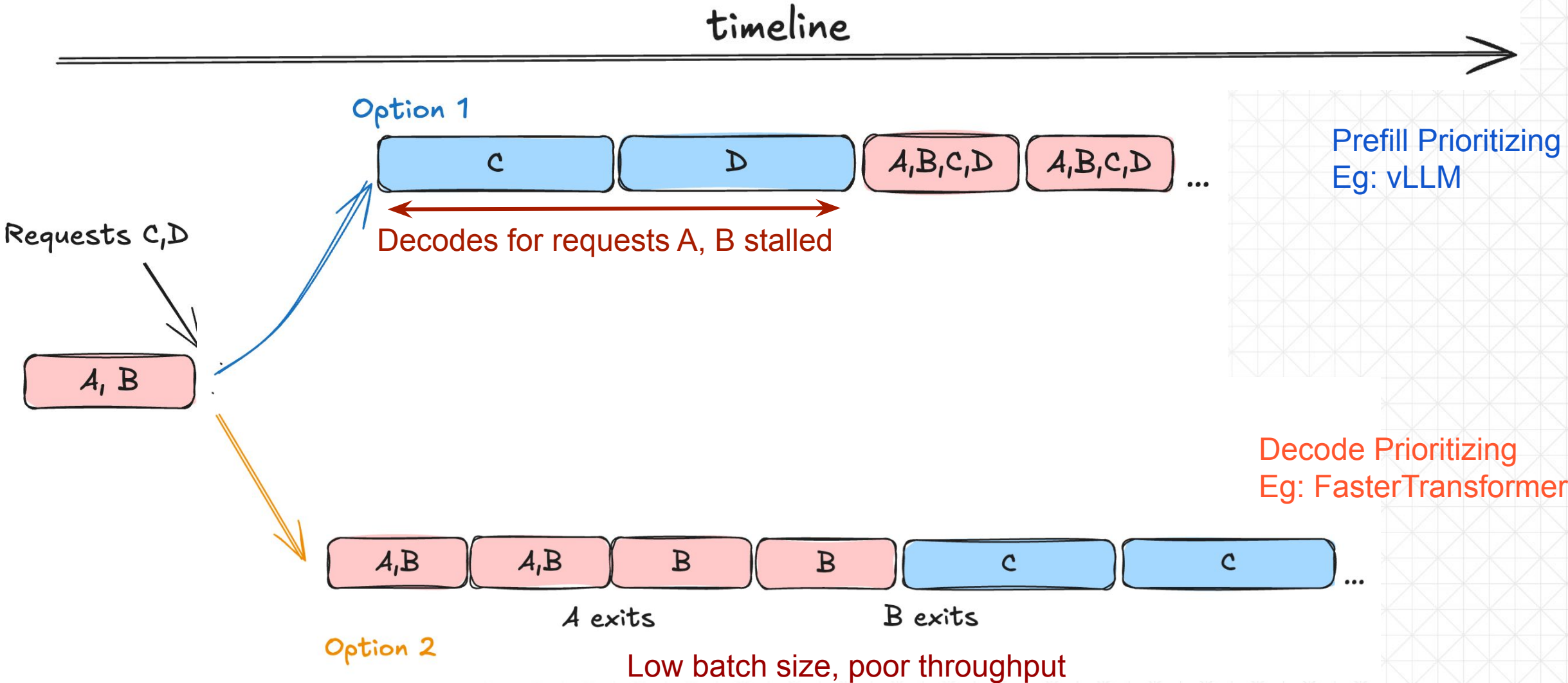


Existing batching methods make a **harsh latency-throughput** tradeoff !

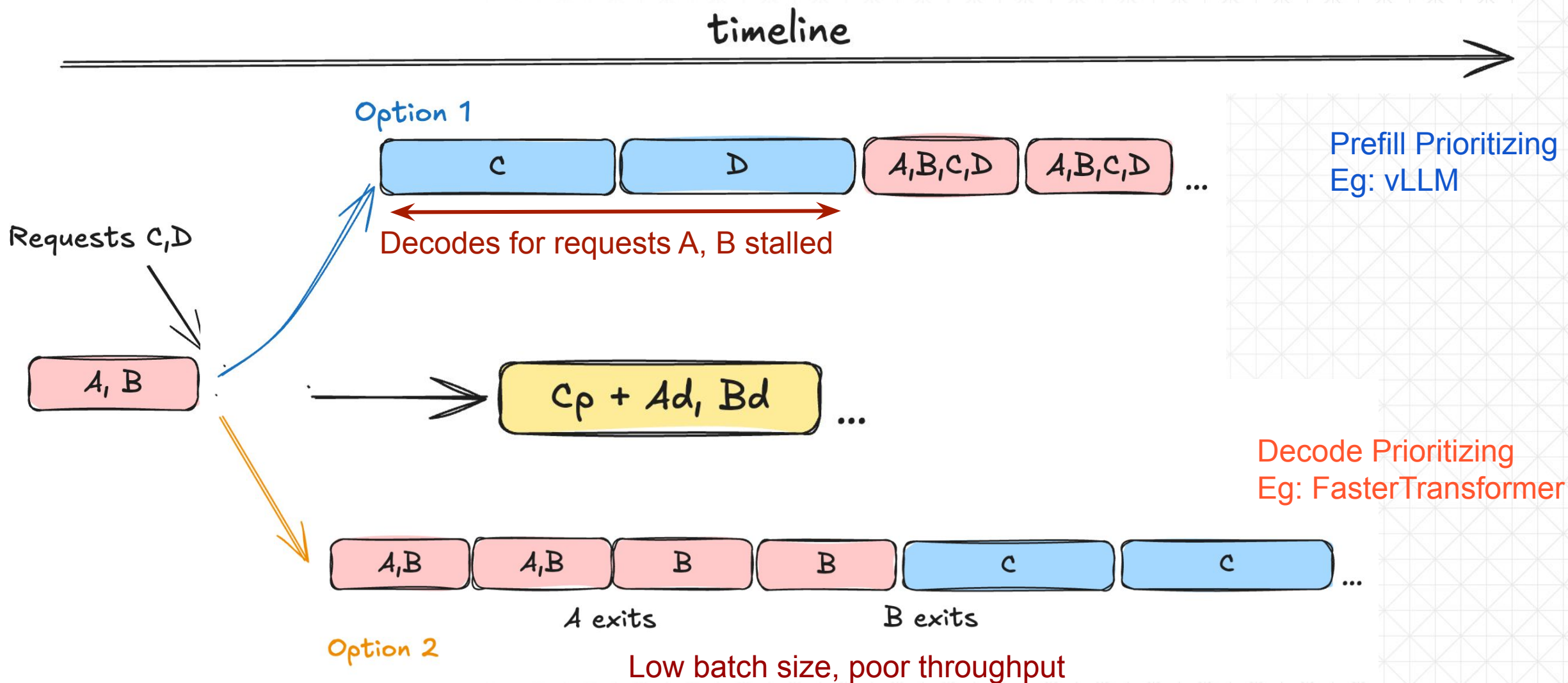


**How can we achieve both high throughput and low-latency ?**

# Batching LLM Inference



# Batching LLM Inference



# Mixed Batching

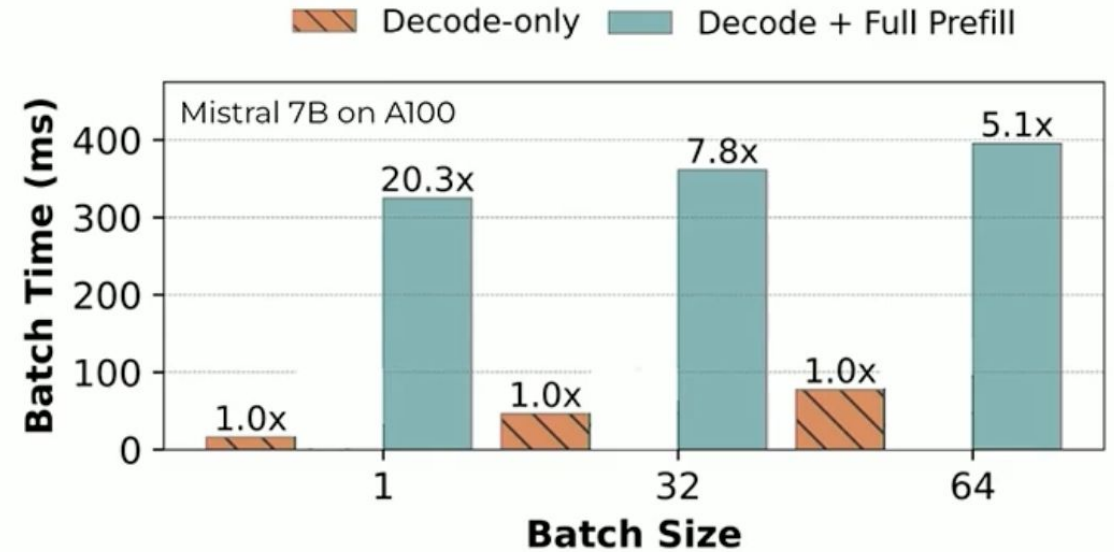
**Idea:** Fused computation of prefill and decode



# Mixed Batching

**Idea:** Fused computation of prefill and decode

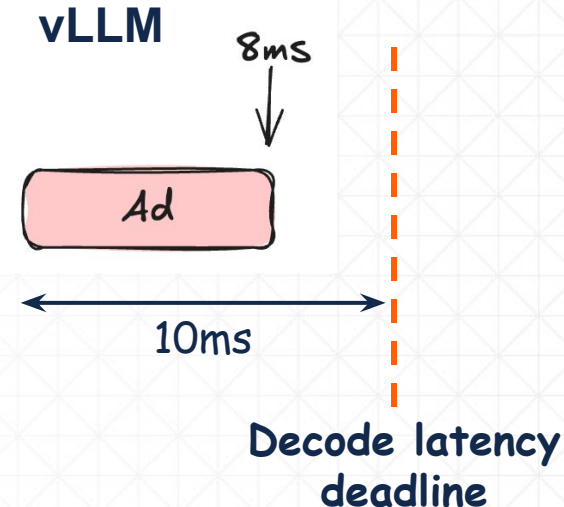
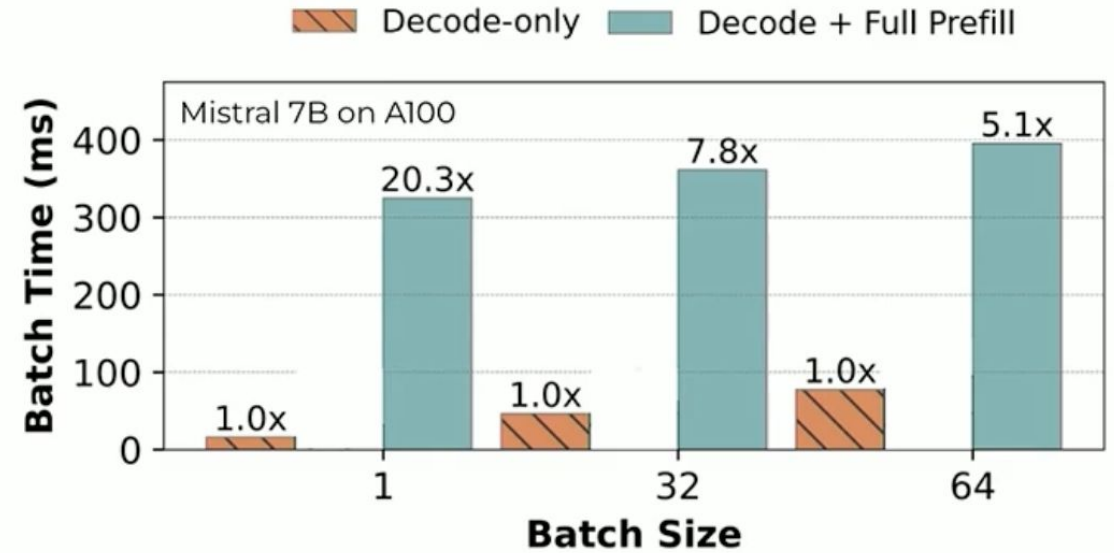
**Challenge:** Naively fuse those steps will lead to increase in latency



# Mixed Batching

**Idea:** Fused computation of prefill and decode

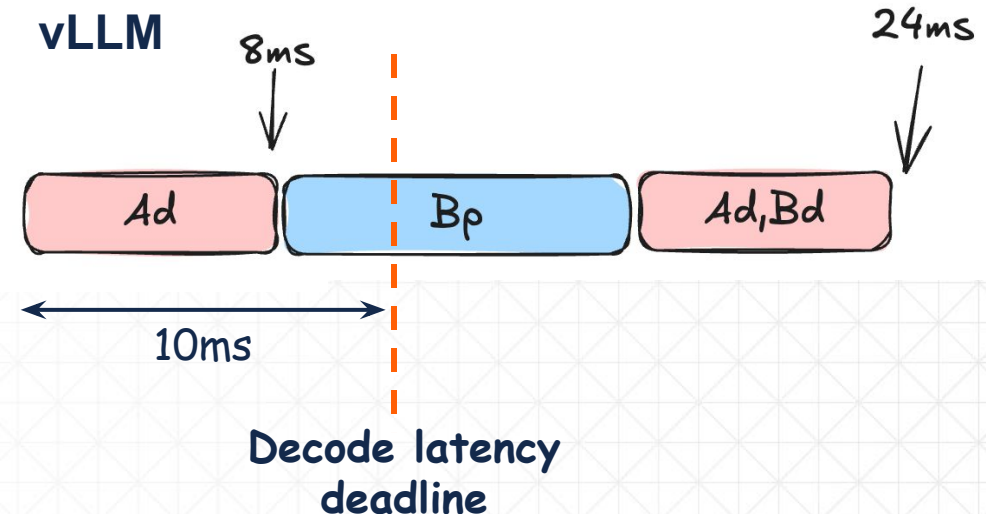
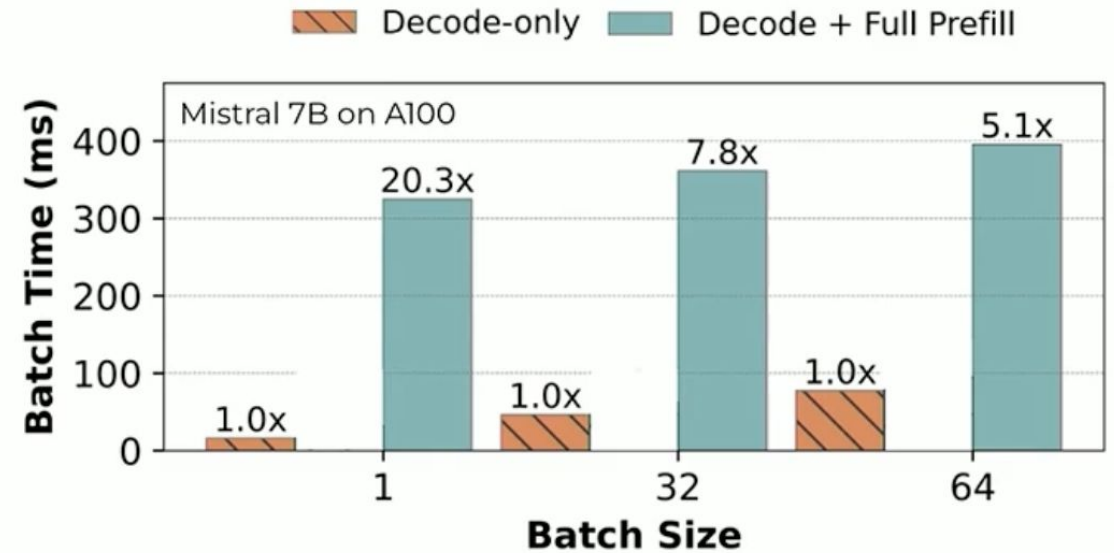
**Challenge:** Naively fuse those steps will lead to increase in latency



# Mixed Batching

**Idea:** Fused computation of prefill and decode

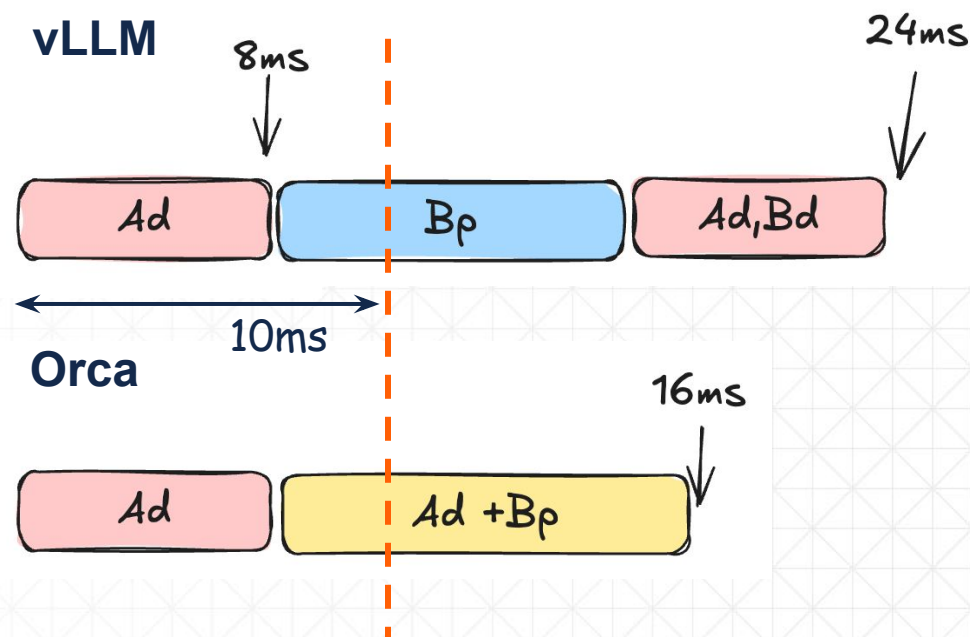
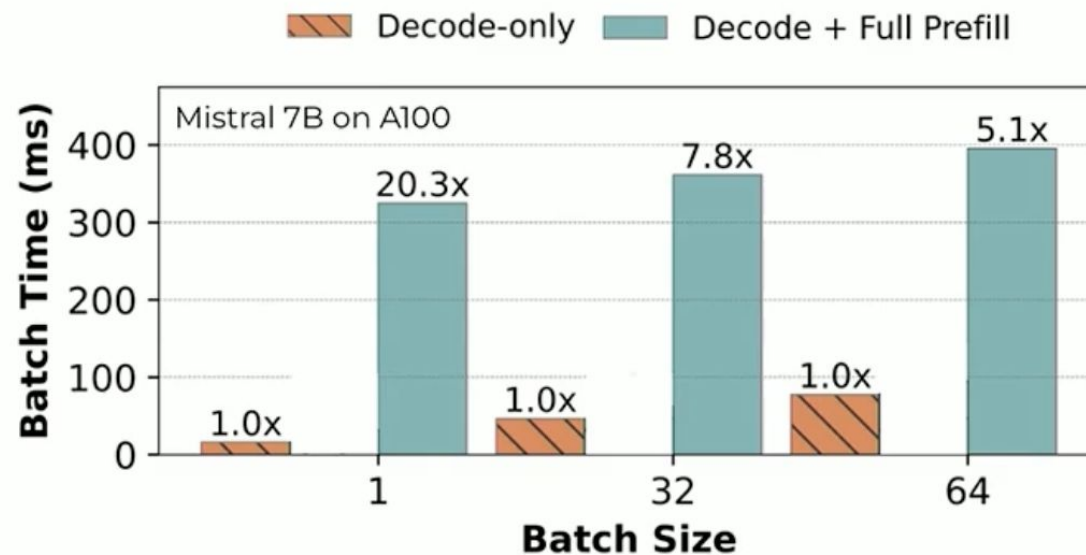
**Challenge:** Naively fuse those steps will lead to **increase in latency**



# Mixed Batching

**Idea:** Fused computation of prefill and decode

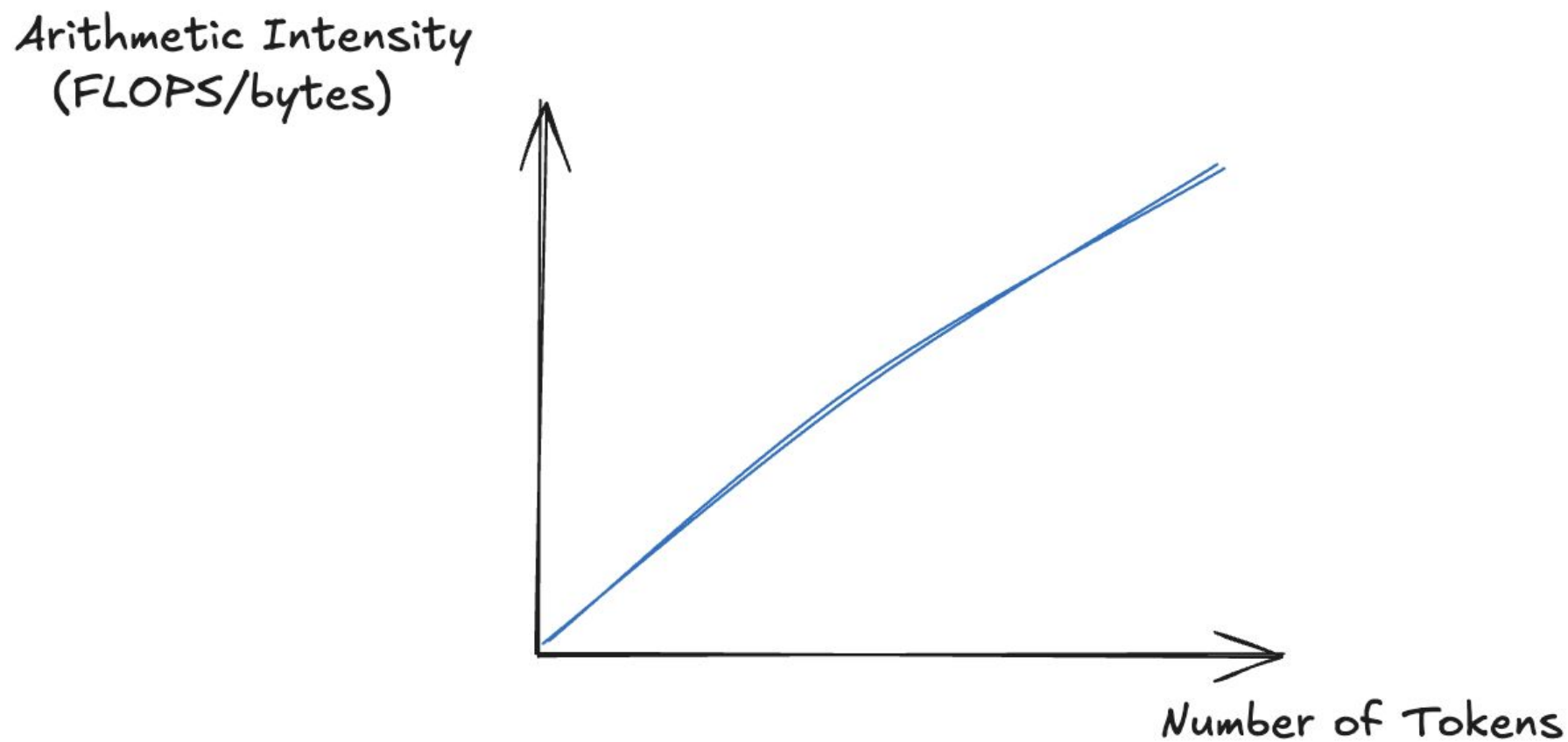
**Challenge:** Naively fuse those steps will lead to **increase in latency**





# Observation: Arithmetic Intensity Slack

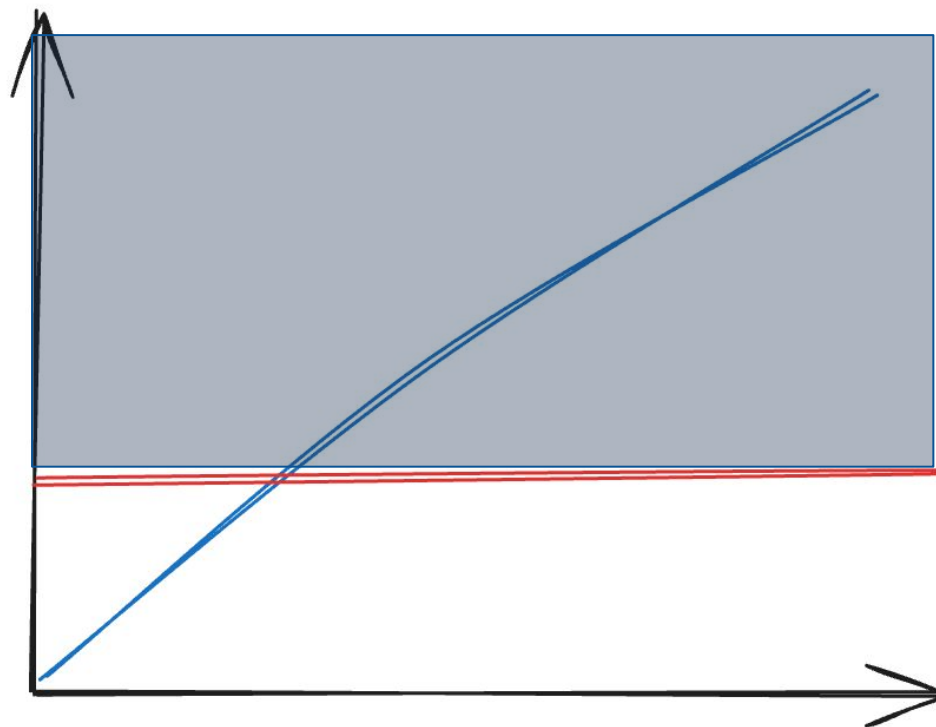
# Observation: Arithmetic Intensity Slack



# Observation: Arithmetic Intensity Slack

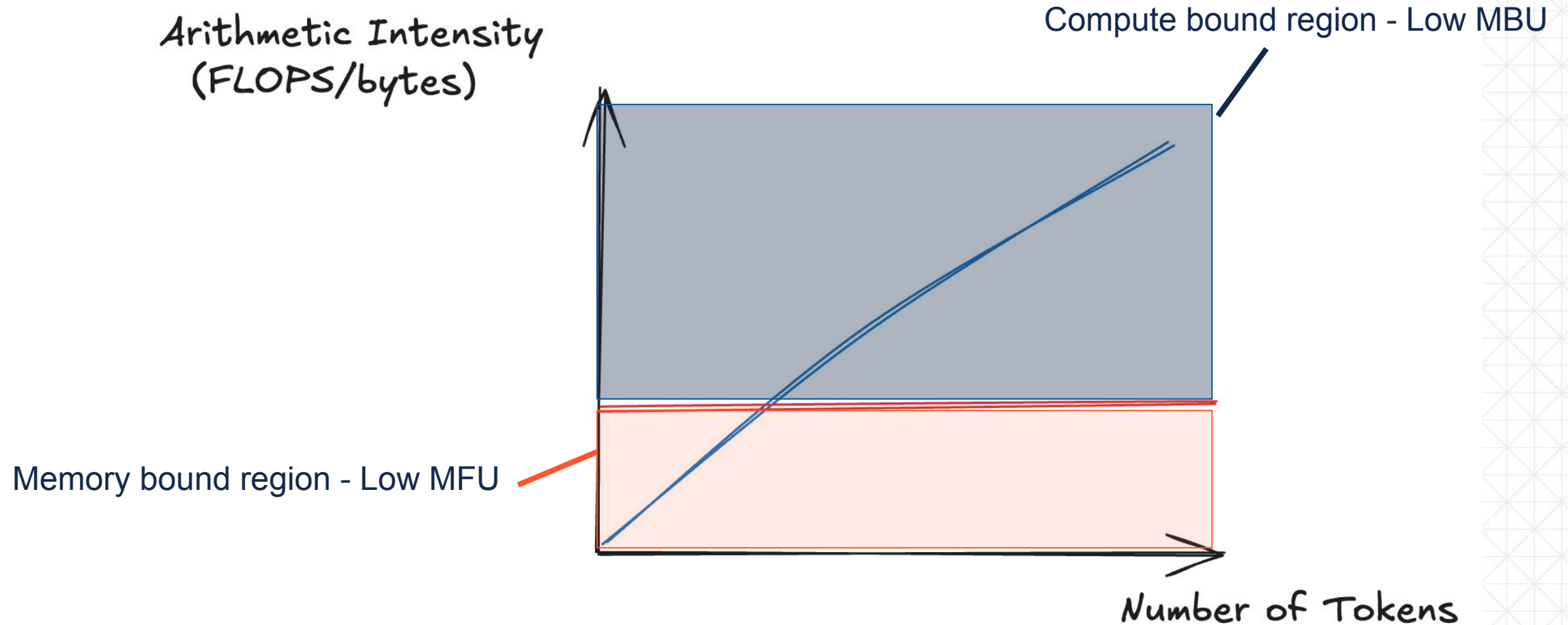
Arithmetic Intensity  
(FLOPS/bytes)

Compute bound region - Low MBU



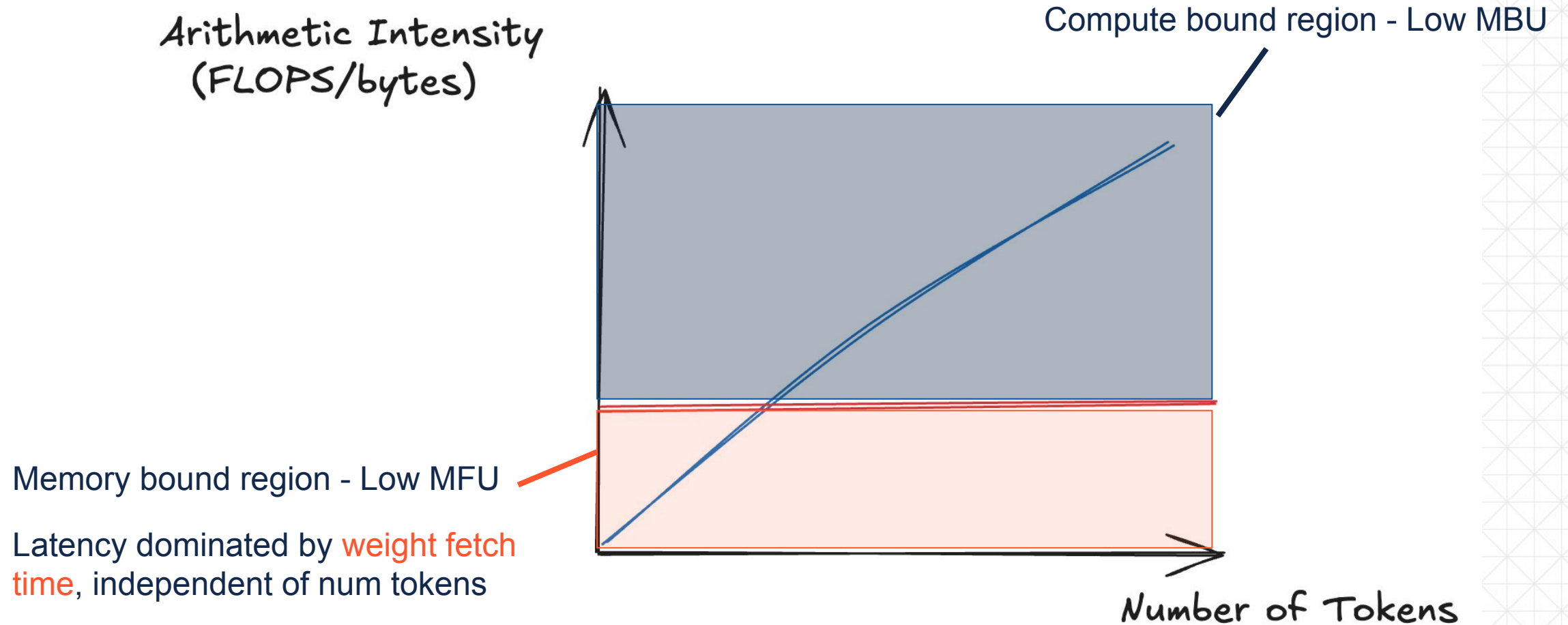
Number of Tokens

# Observation: Arithmetic Intensity Slack

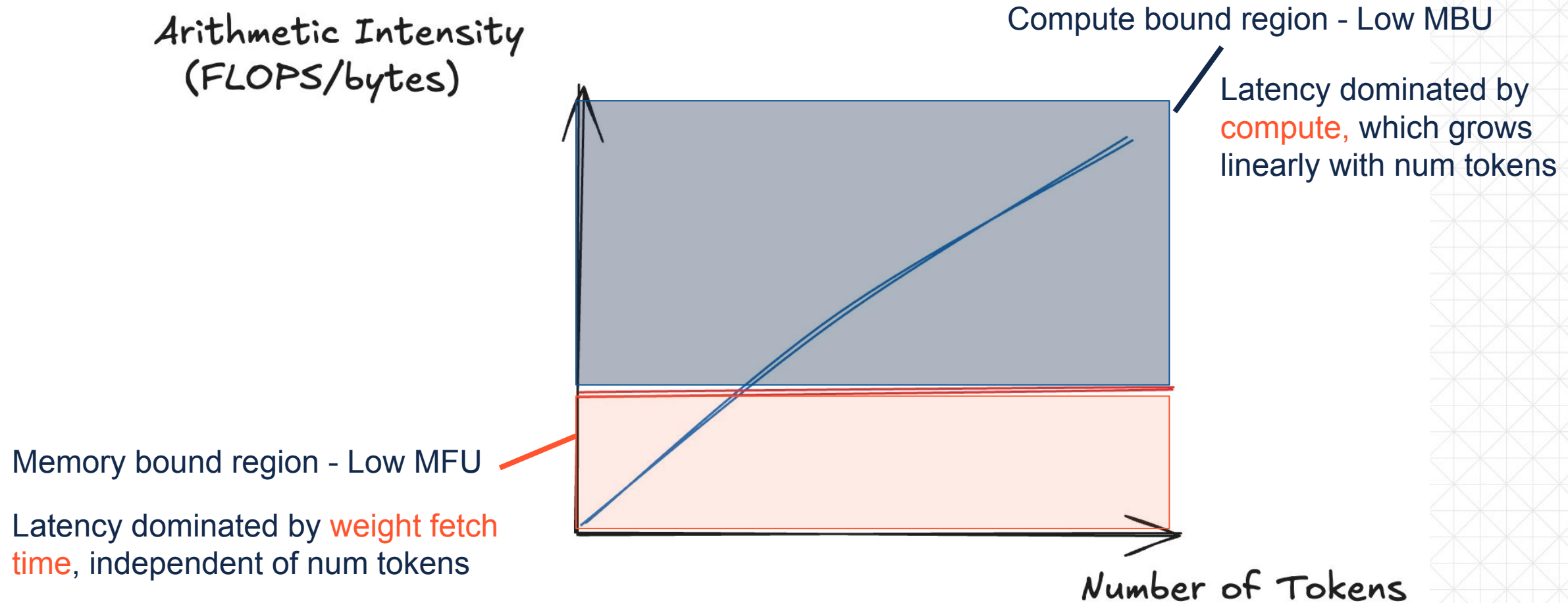




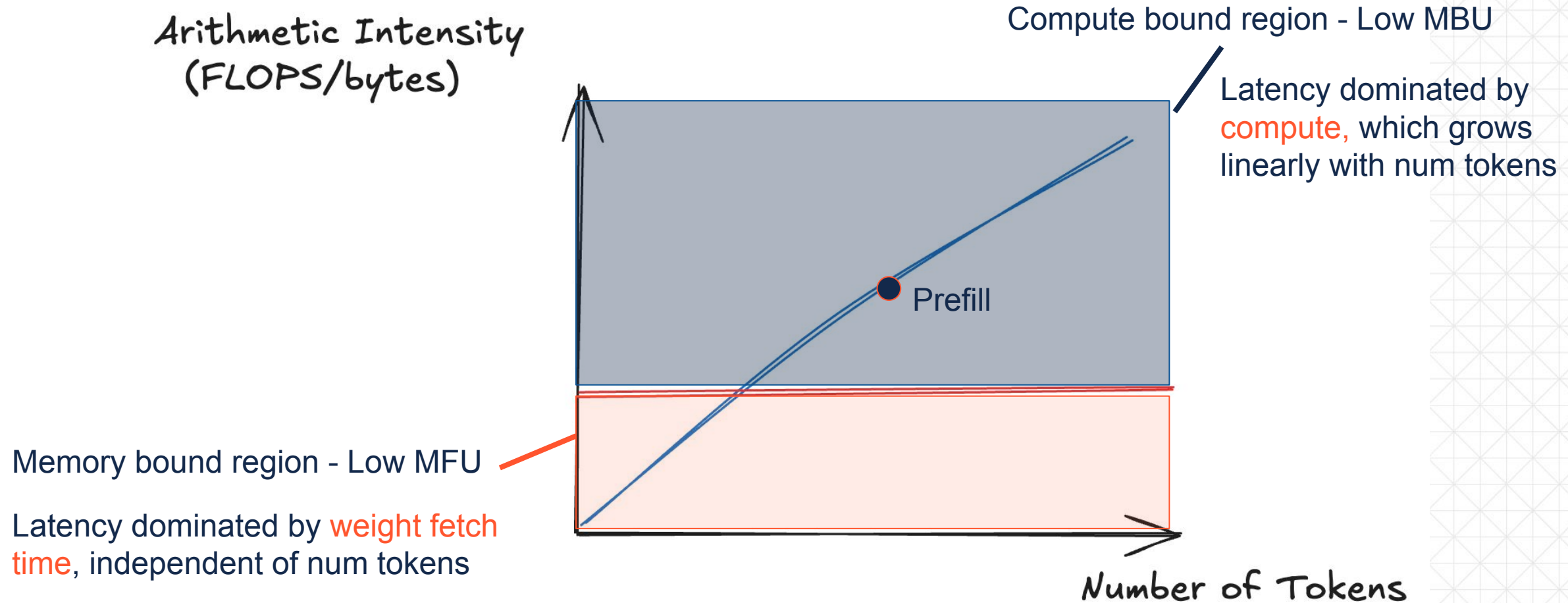
# Observation: Arithmetic Intensity Slack



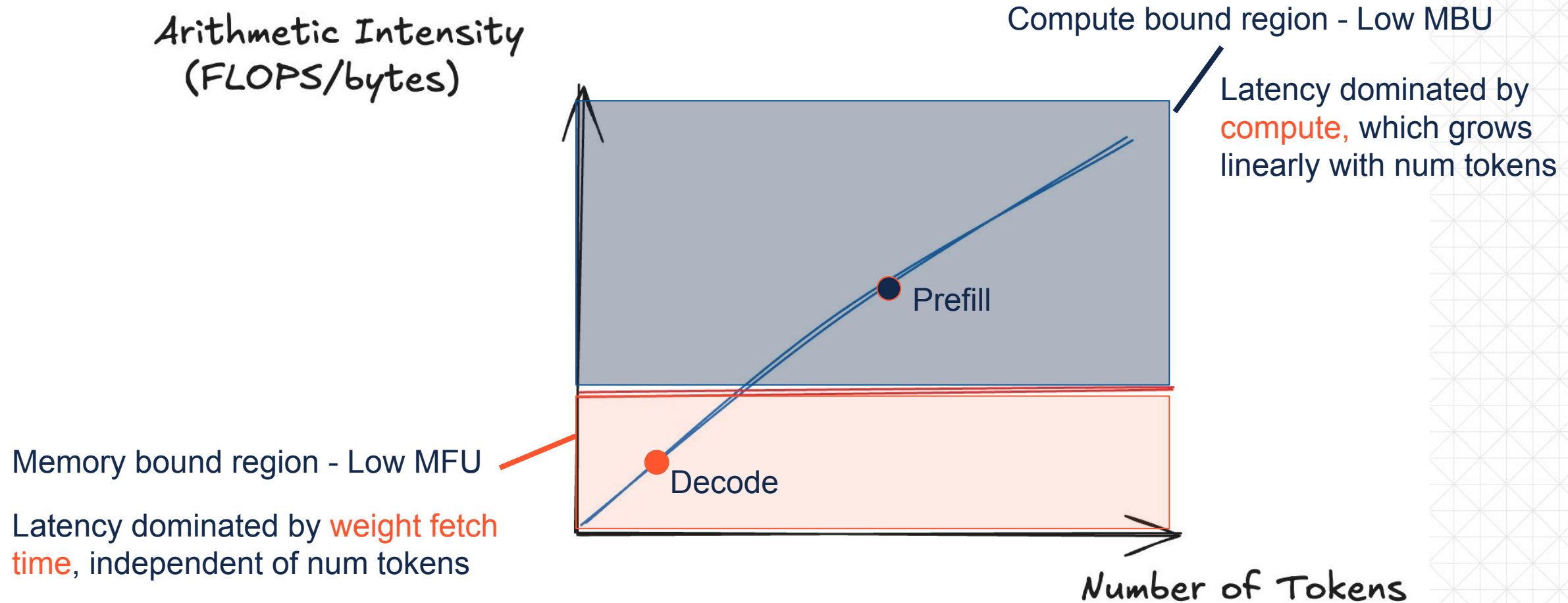
# Observation: Arithmetic Intensity Slack



# Observation: Arithmetic Intensity Slack

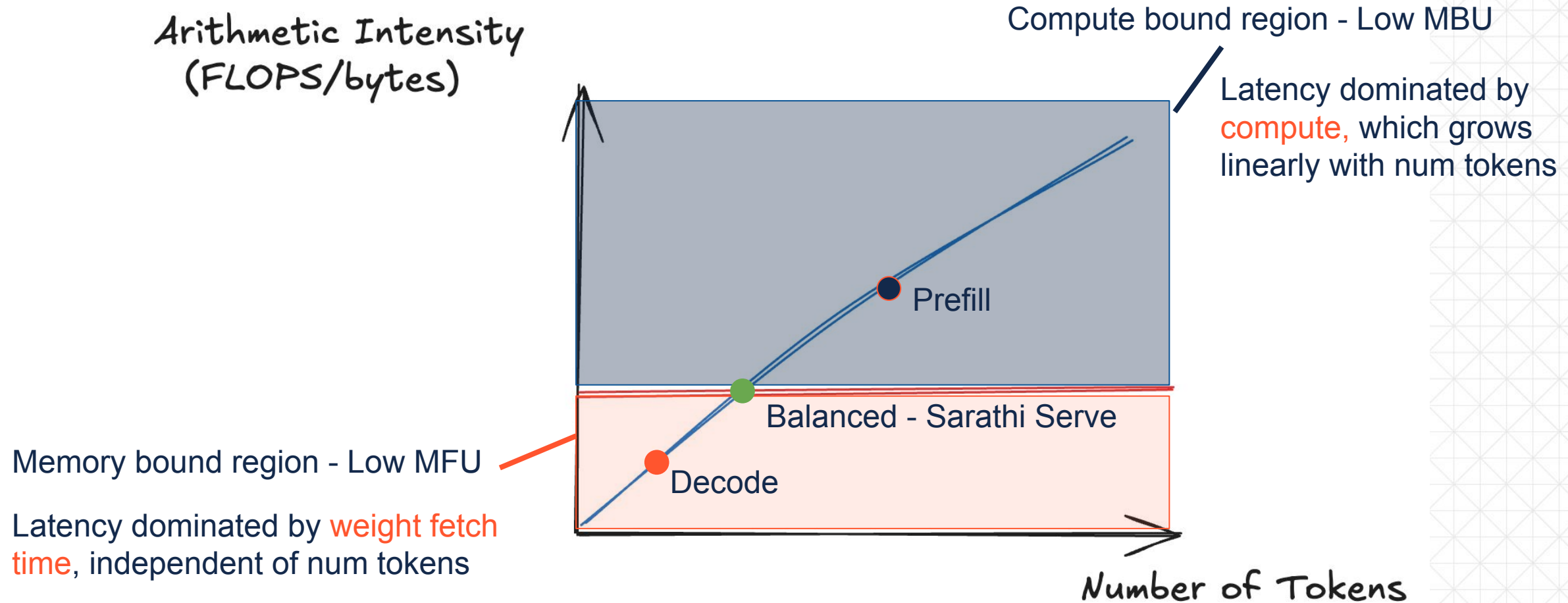


# Observation: Arithmetic Intensity Slack





# Observation: Arithmetic Intensity Slack



# Stall-free Batching

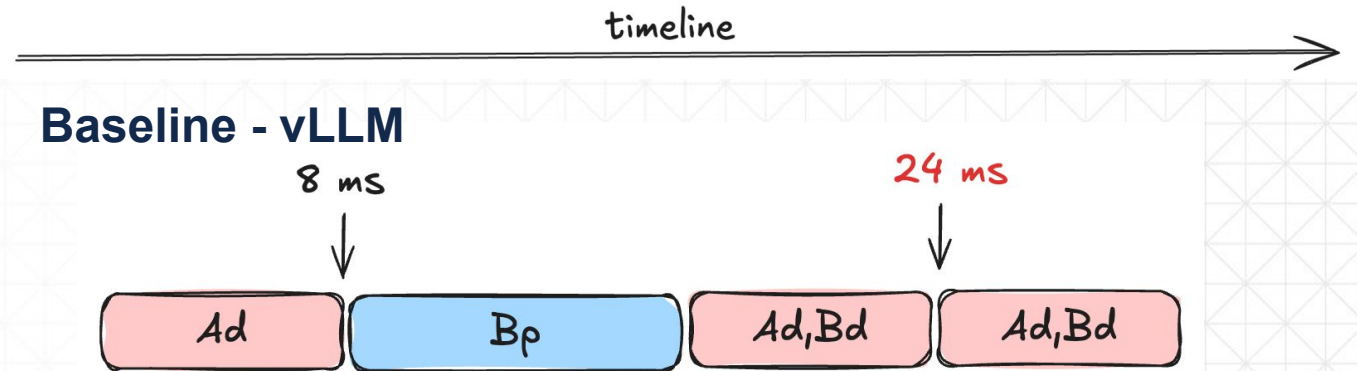
## Key Idea

Split large prefill into smaller chunks - just enough to consume the leftover compute budget in decode batches

# Stall-free Batching

## Key Idea

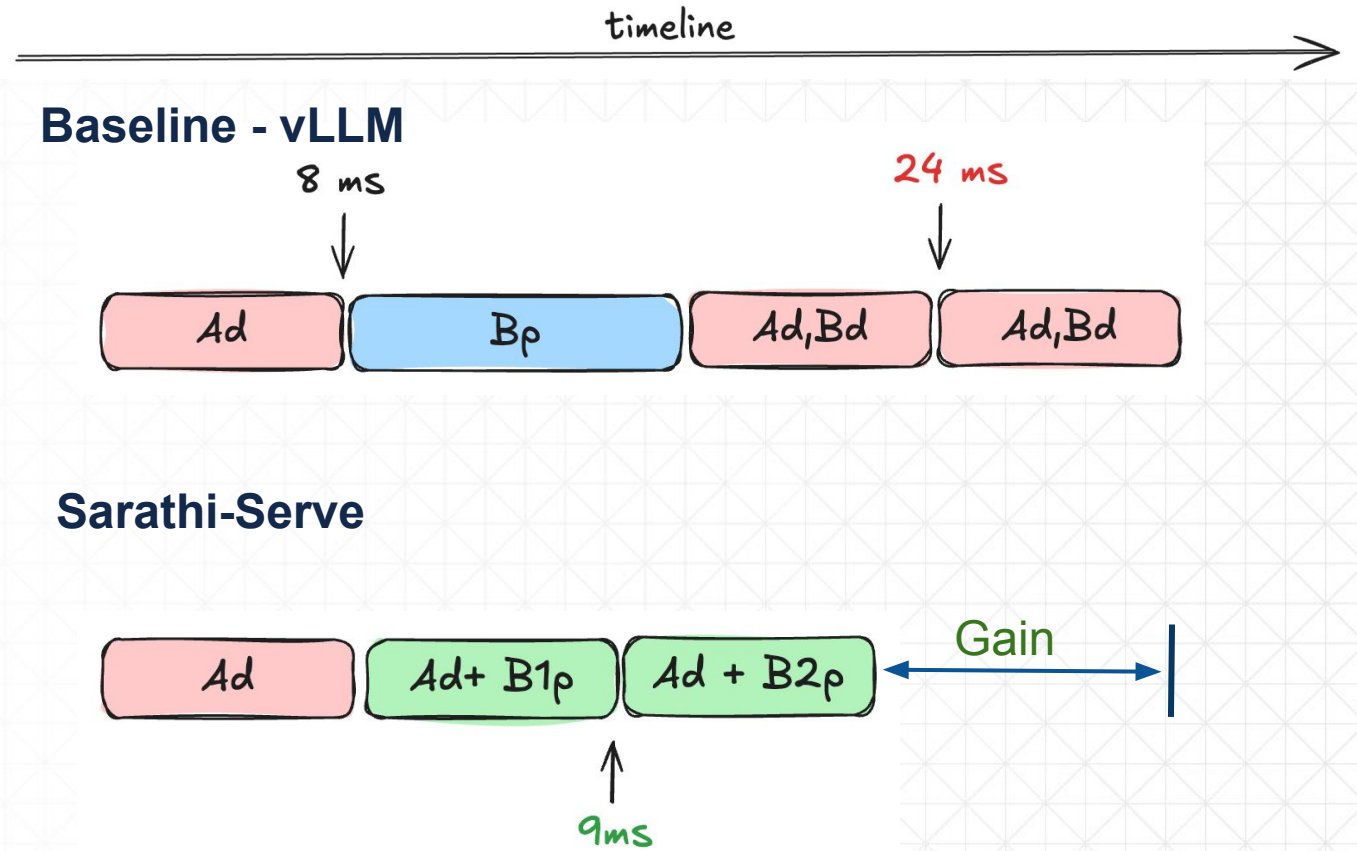
Split large prefill into smaller chunks - just enough to consume the leftover compute budget in decode batches



# Stall-free Batching

## Key Idea

Split large prefill into smaller chunks - just enough to consume the leftover compute budget in decode batches



~ **Uniform inter-token latency** based on TBT SLO



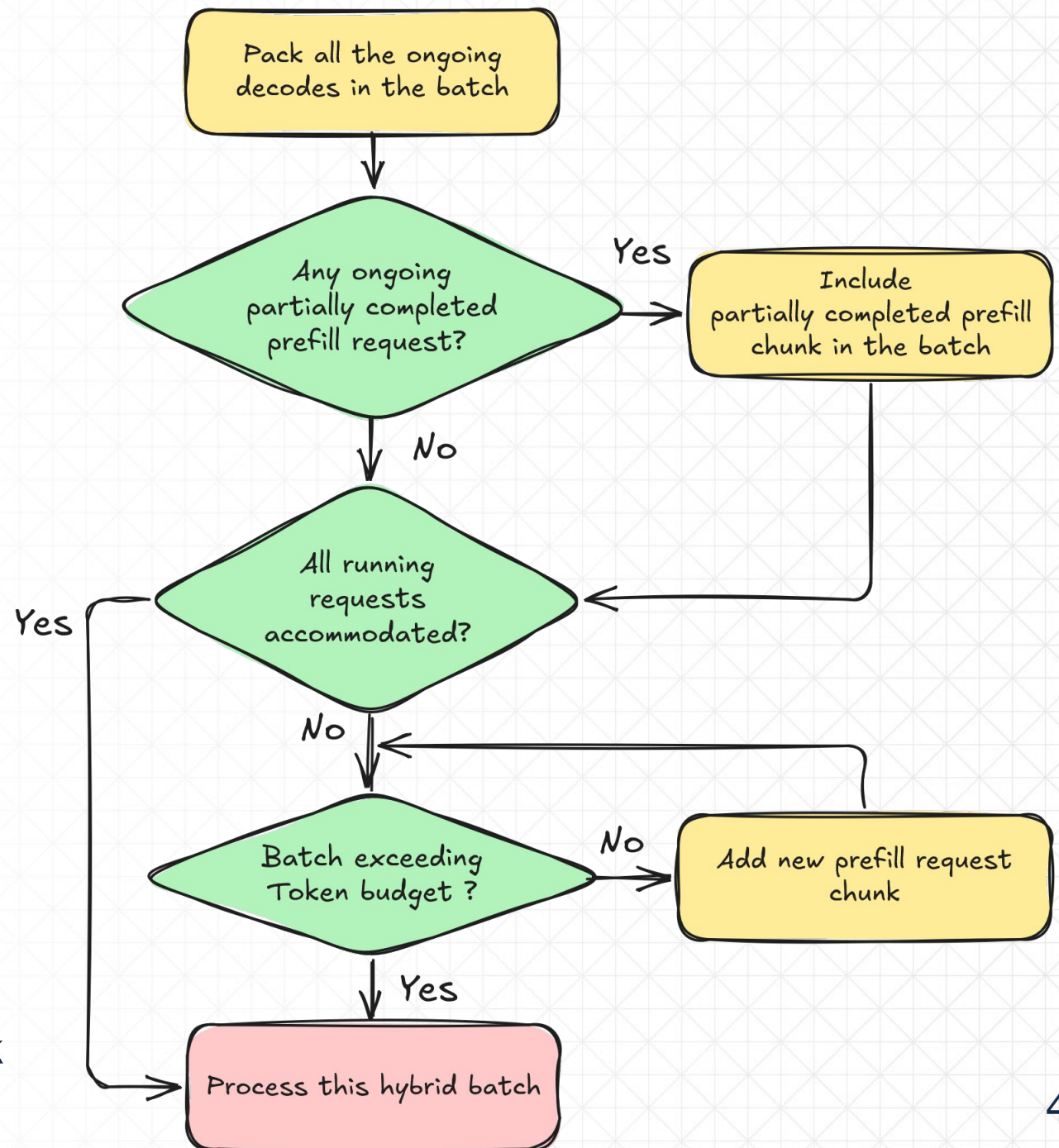
# Stall-free Batching

1. First the attention batch is filled with **ongoing decode** tokens
2. Optionally one **ongoing prefill** chunk
3. Finally, **new requests** are added

## Sarathi-Serve



→ Decodes never experience a generation stall of running prefill chunk



# Evaluation Results: Metrics

**Time to first token (TTFT):** Time required for the first token to show up from the time user submits a request

**Time between tokens (TBT):** Latency between each output token, also called TPOT

**Capacity:** Maximum QPS that can be served while satisfying latency SLOs

# Evaluation Results

## Setup

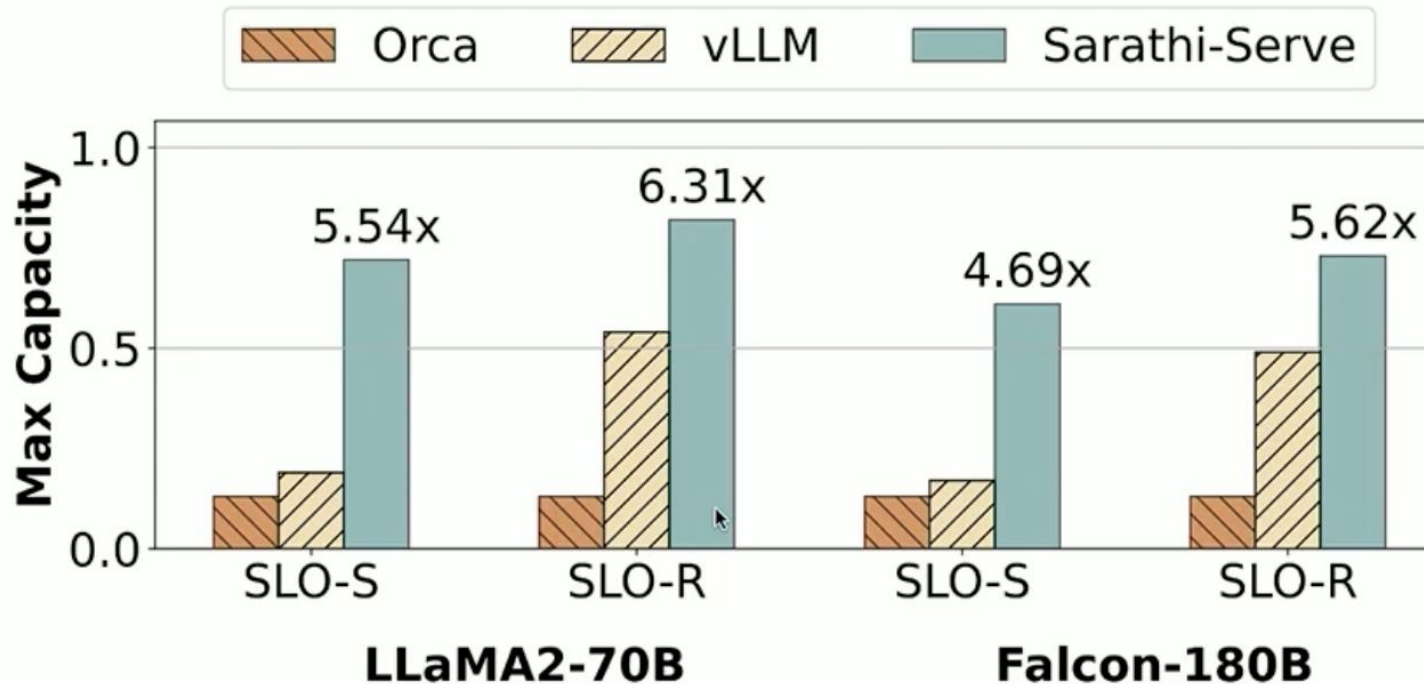
ShareGPT4 trace on A100 GPUs with strict(S) and relaxed (R) latency SLOs



# Evaluation Results

## Setup

ShareGPT4 trace on A100 GPUs with strict(S) and relaxed (R) latency SLOs



- **Strict** latency SLO: split prompts into smaller chunks → lower tail latency
- **Relaxed** latency SLO: increase the token budget → allow more efficient prefills



# Key takeaways

**Problem:** State-of-the-art systems sacrifice decode latency to achieve higher throughput

**Key insight:** Low arithmetic intensity of decodes allows for adding compute intensive prefills with negligible decode latency cost

**Key results:** Sathari-Serve achieve optimal performance in both latency and throughput simultaneously leading up to 6x higher capacity under SLO constraints

# Strength vs Limitations

## Strength

- Chunked-prefill splits long prompts across iterations to balance prefill and decode, reducing idle GPU cycles and pipeline bubbles.
- Stall-free scheduling inserts prefill chunks into decode batches without pauses, improving throughput while keeping latency low.

## Limitations

- Equal-compute chunking may require padding or overhead when splitting very short prompt prefills, wasting compute compared to single-shot processing and lead to inefficiencies.
- Accounts only for a single GPU scenario, did not account for the network bound latency in multiple GPUs scenarios.

# DistServe

Disaggregating Prefill and Decoding for Goodput-optimized Large  
Language Model Serving

Presenters: Minseob Shin and Neil Deo



UNIVERSITY OF  
**ILLINOIS**  
URBANA-CHAMPAIGN



# Table of Contents

1. **Background and Motivation**
2. **Computational analysis of Prefill and Decode**
3. **DistServe Implementation**
4. **Evaluation and Impact**
5. **Summary**



# Background

Existing LLM serving systems co locate the two phases and batch the computation of prefill and decoding across all users and requests

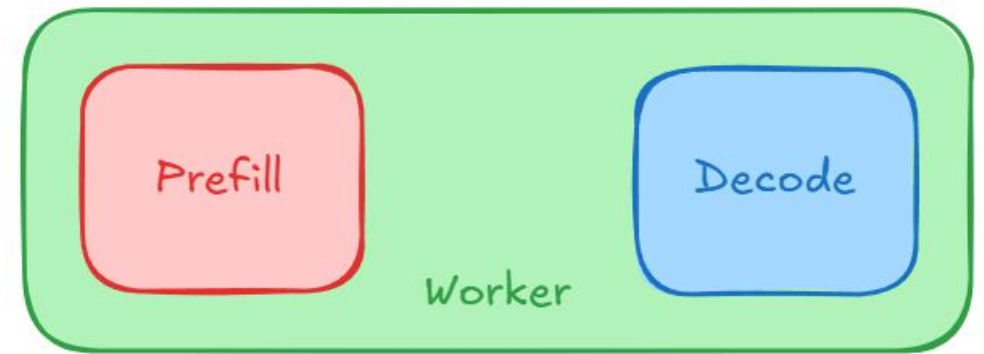
Key Metrics

**TTFT** - time to first token

**TPOT** - time per output token

**Goodput** = # of completed requests *that meet the SLO* per second

**SLOs** - service-level objectives



# Background

Previous approaches to optimize LLM serving includes **Continuous batching** and **Model parallelism**.

Problems:

Prefill-decoding interference

Ineffective scheduling

Resource and parallelism coupling



**SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills**

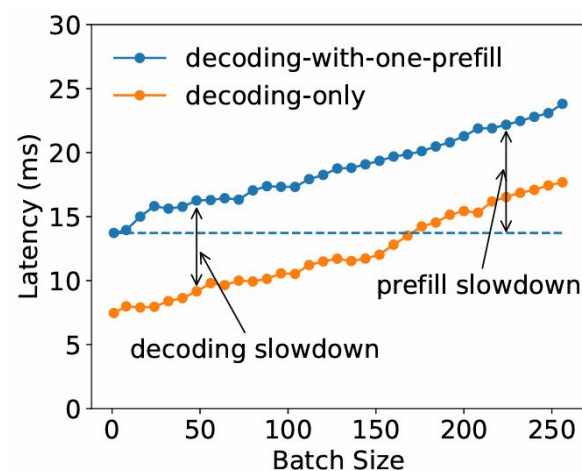
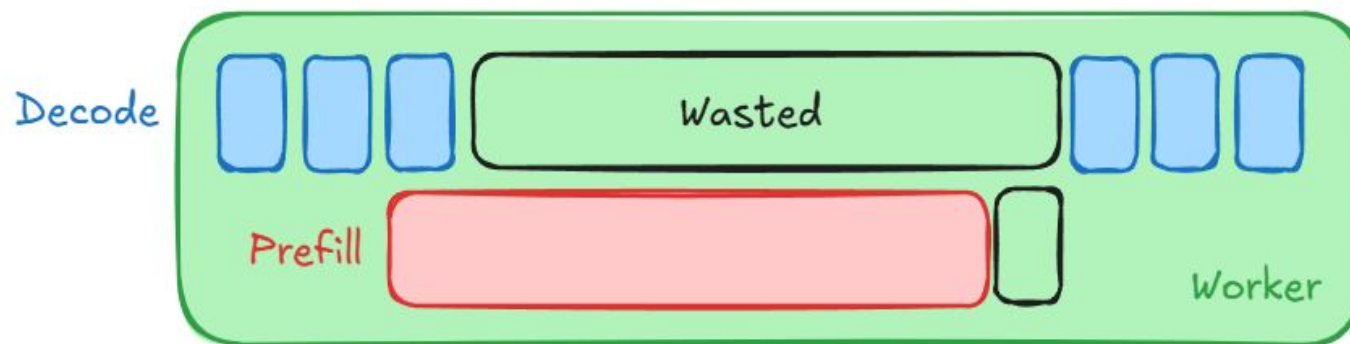
**Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve**

# Continuous Batching

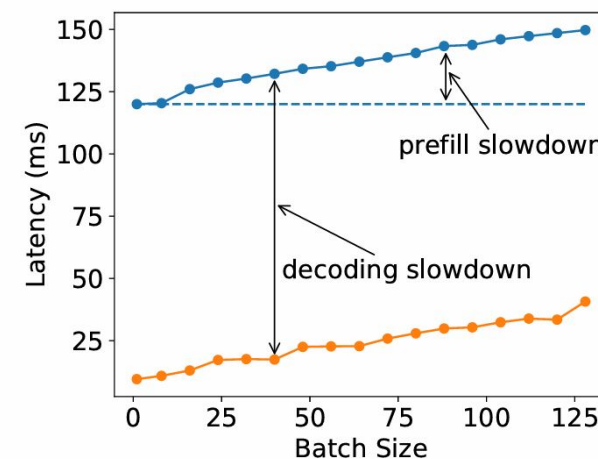
batches the prefill of new requests with the decoding of on going ones.

GPU utilization and maximizes the overall system throughput

causes **Prefill-Decoding Interference**



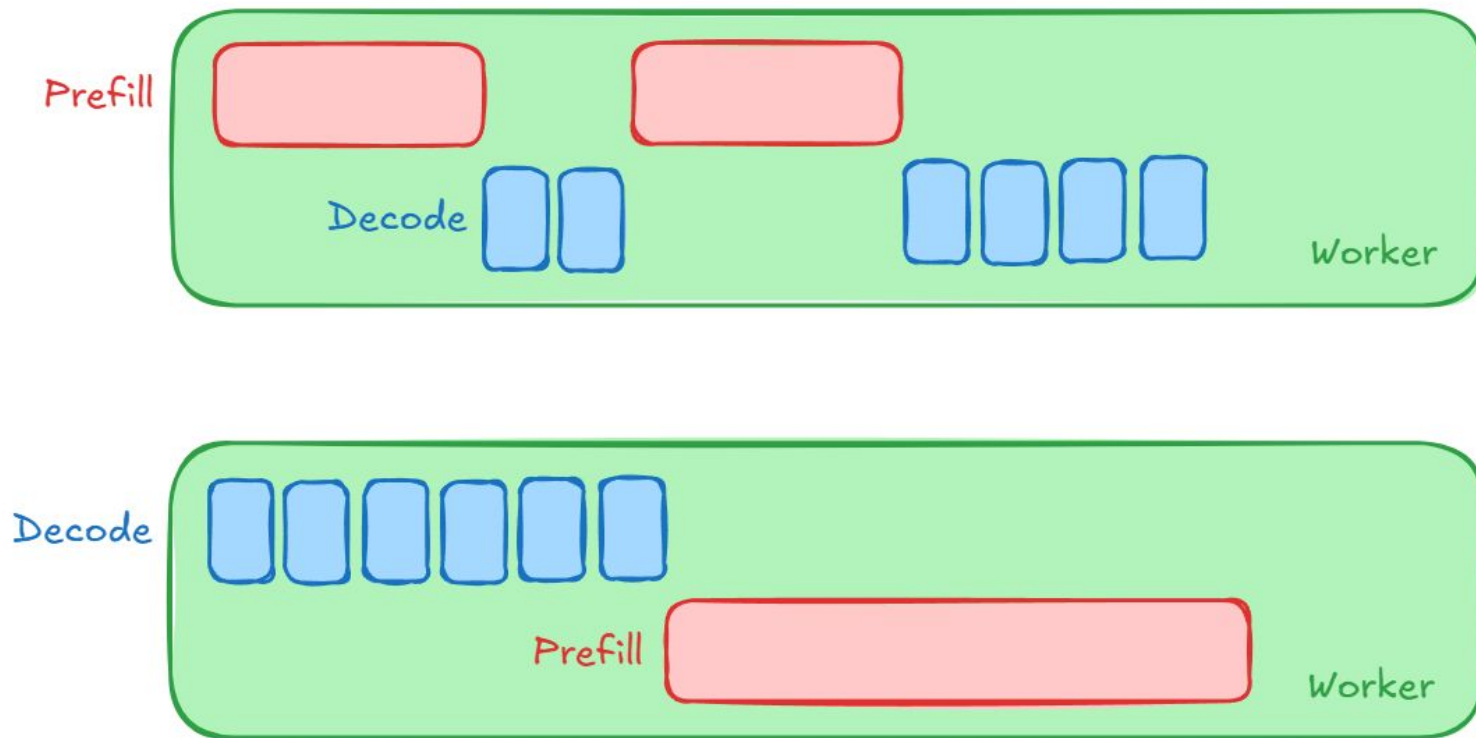
(a) Input length = 128



(b) Input length = 1024

# Ineffective Scheduling

Prioritizing tasks in either phase negatively affects the latency of the other, rendering priority scheduling ineffective.

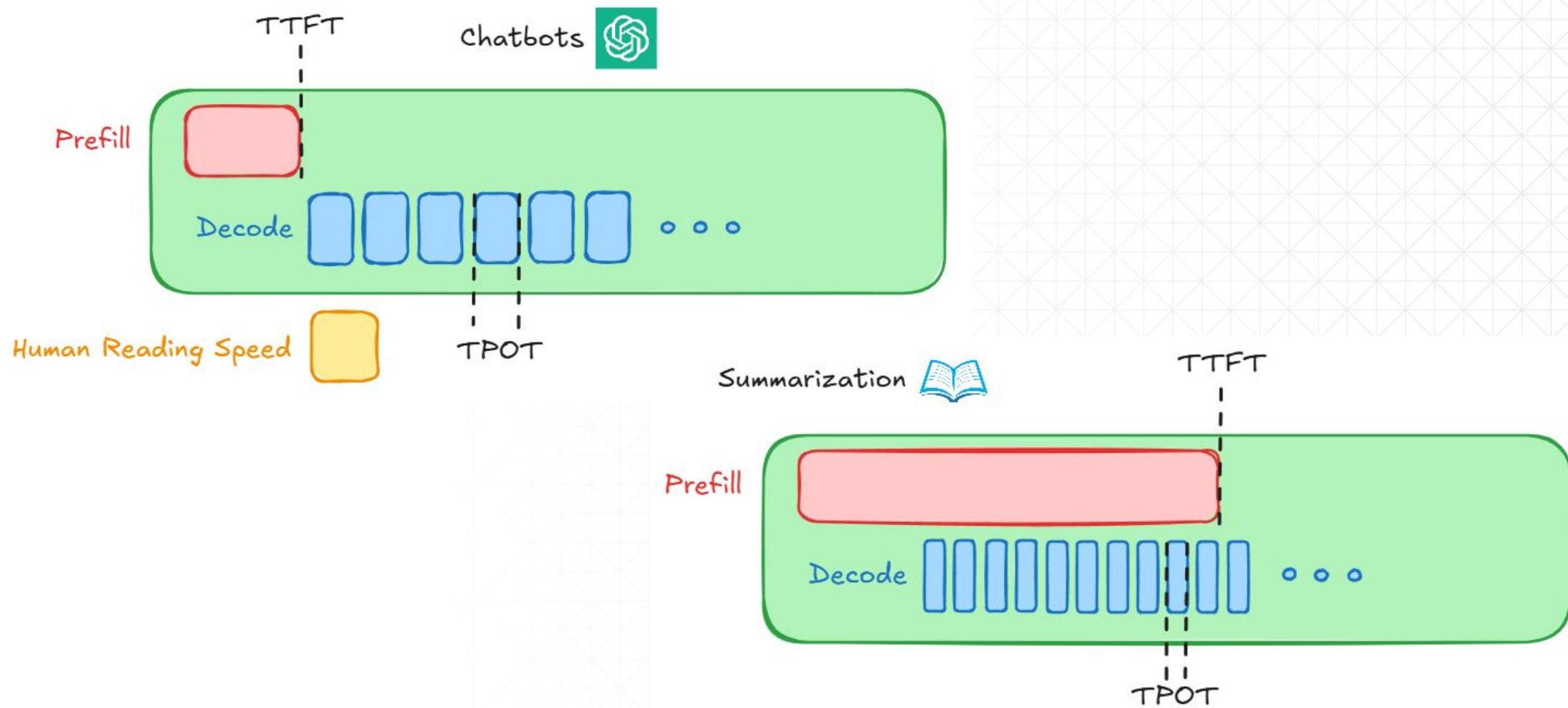




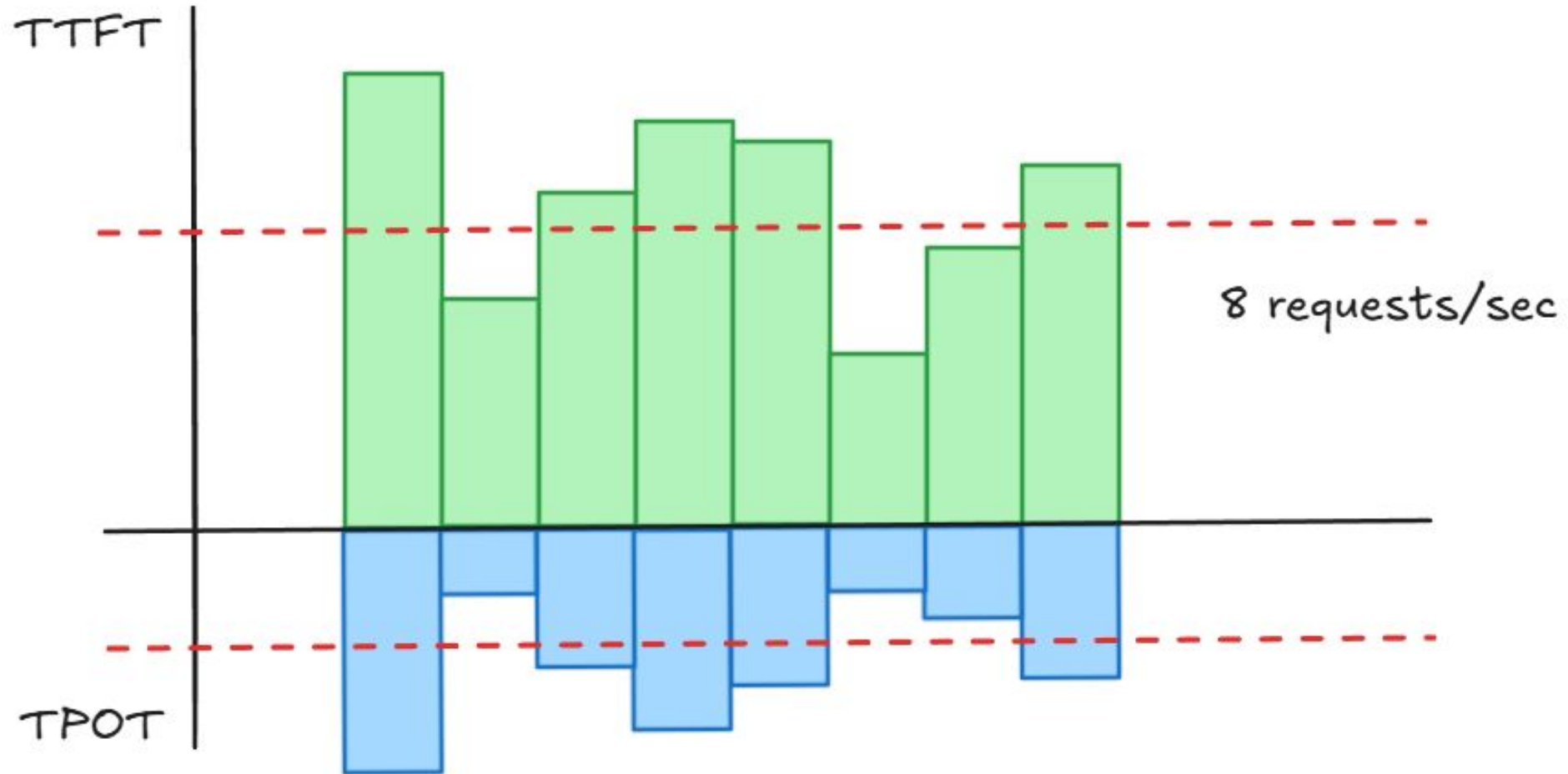
# Why Goodput Matters: Beyond Throughput and Latency

- Captures both service quality and efficiency
- Higher per-GPU goodput directly translates into lower cost per query

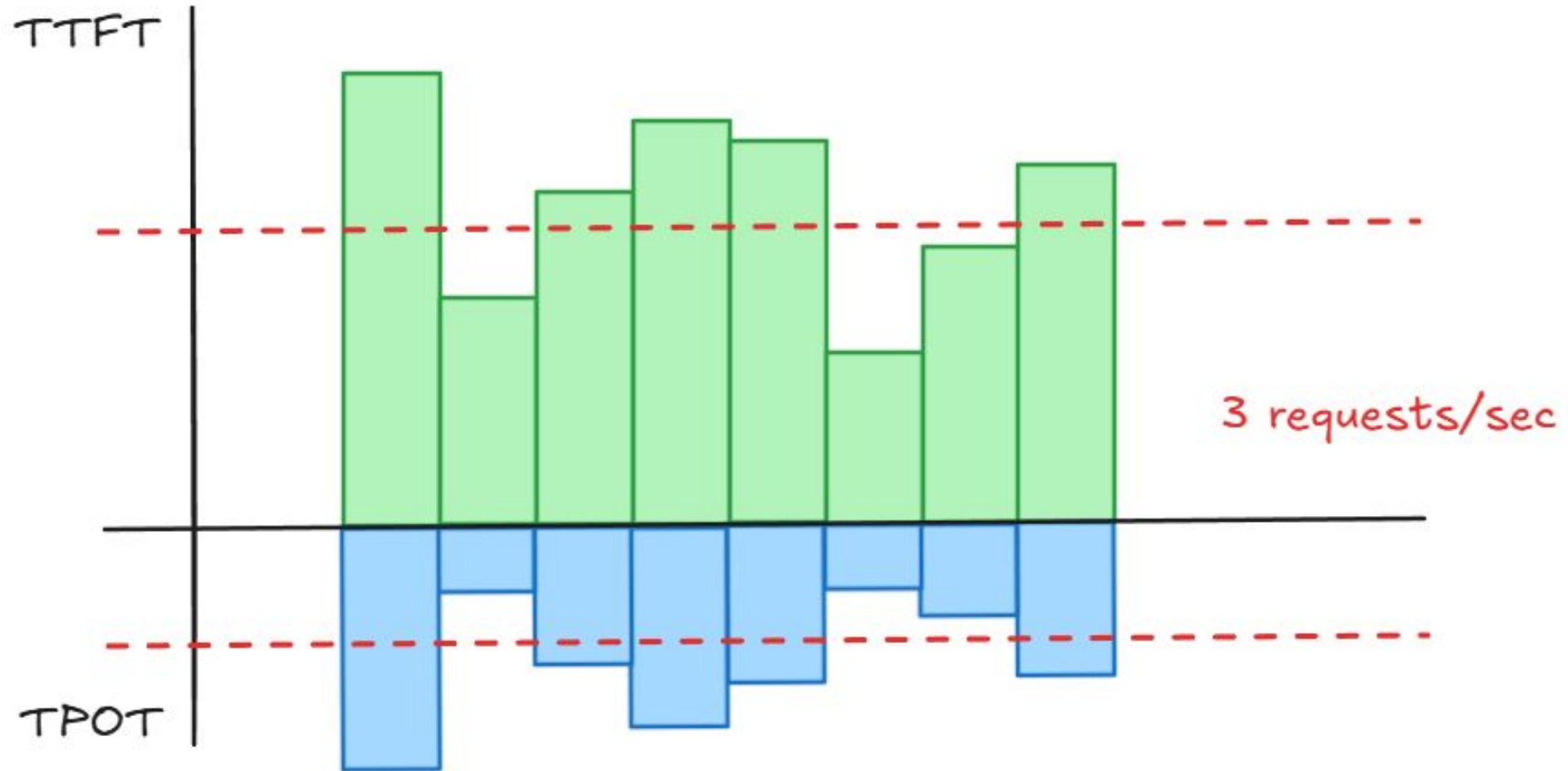
# Applications have diverse SLO



# High Throughput $\neq$ High Goodput



# High Throughput $\neq$ High Goodput

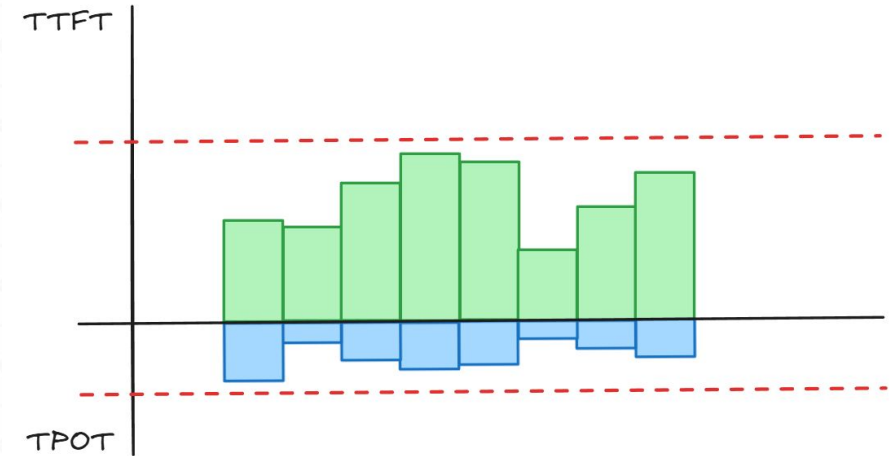
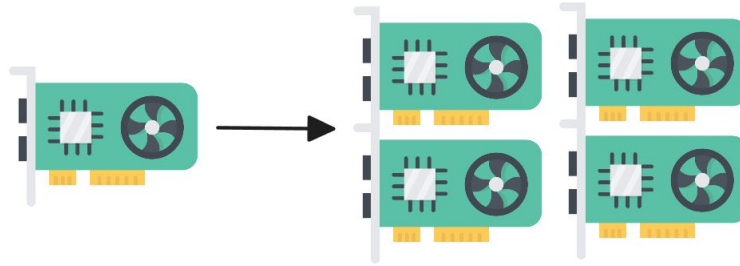




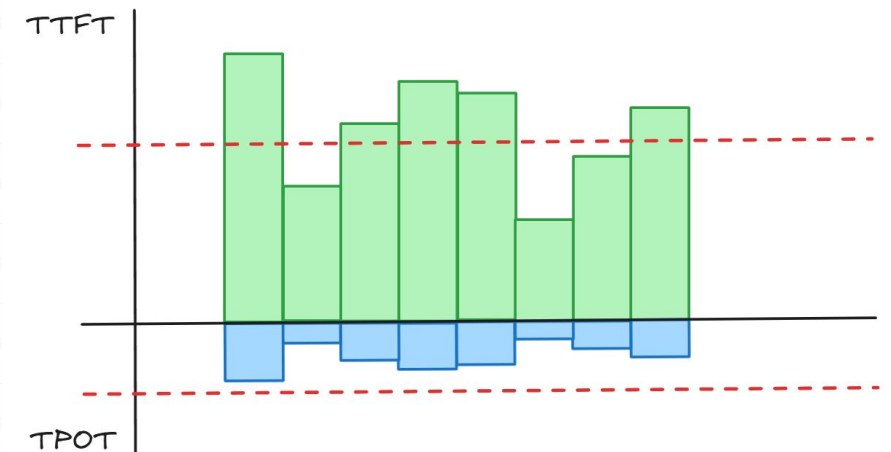
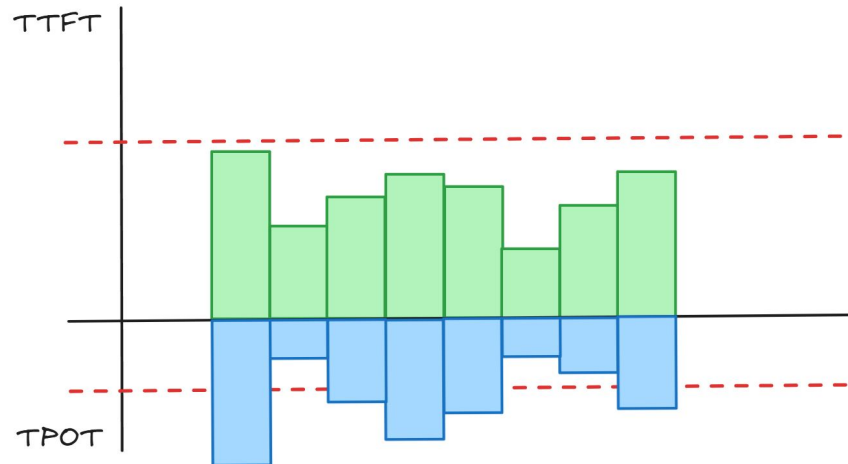
**How can we achieve high goodput?**

# How can we achieve high goodput?

## Overprovision resources



## Model parallelism



# But at what cost?

Higher Cost for Additional Resources



Higher Cost

Resource and parallelism coupling

Prefill

Compute-bound

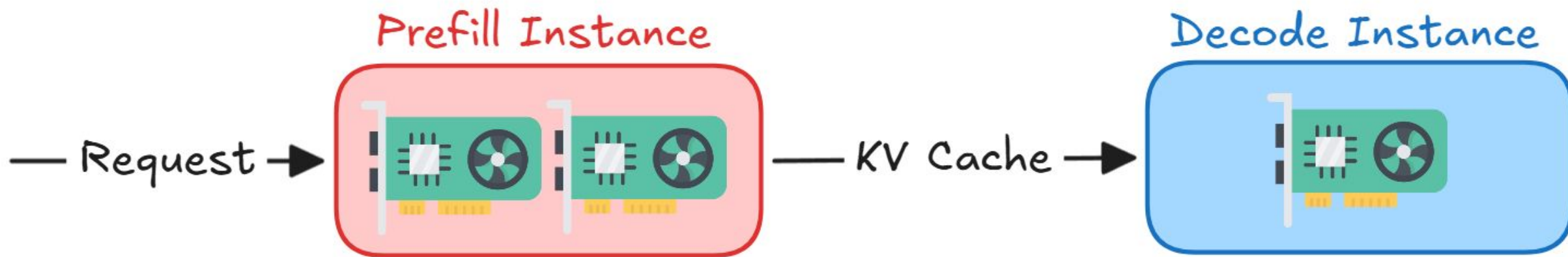
Decoding

Memory-bound

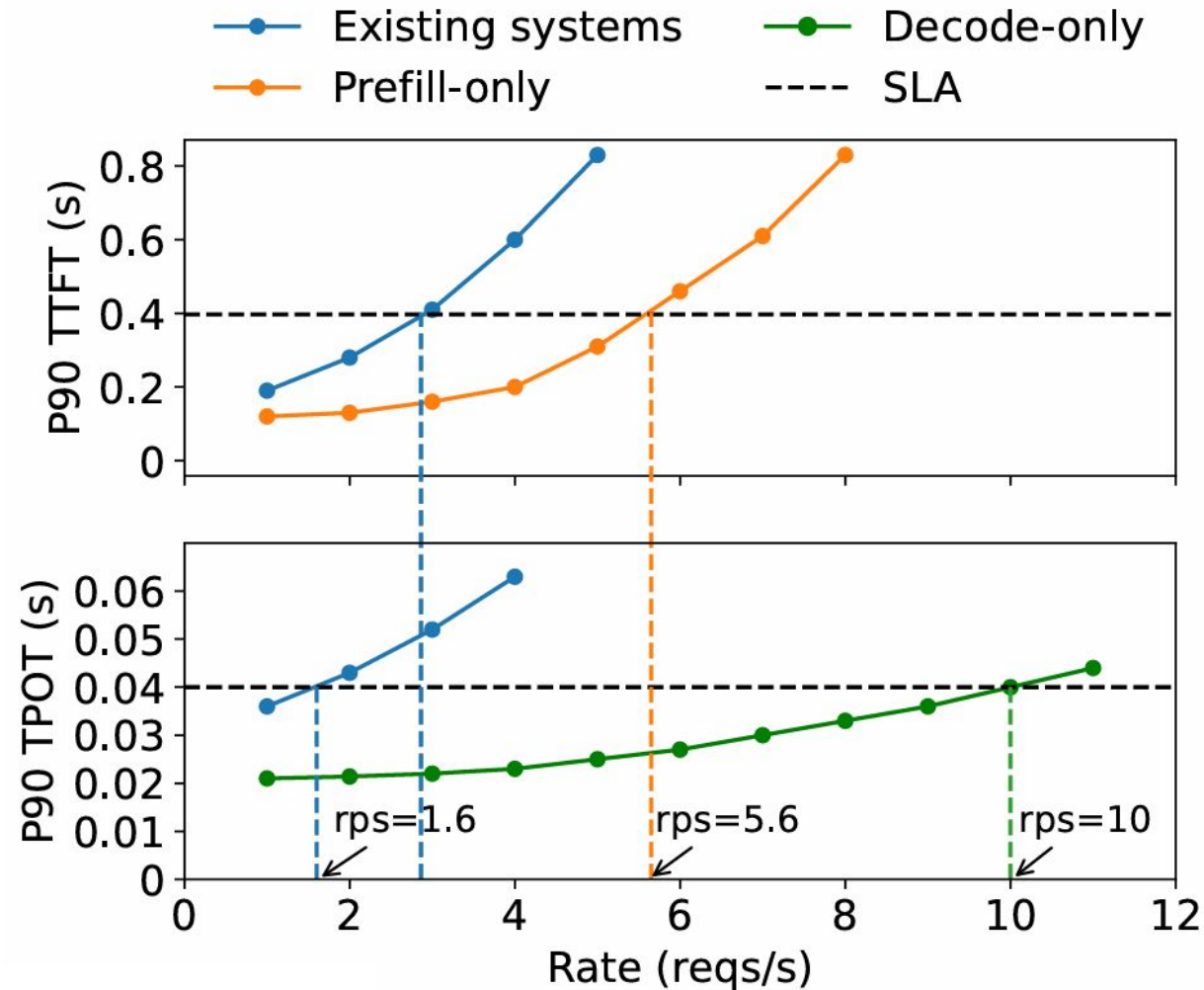
# Disaggregating Prefill and Decode



# Disaggregating Prefill and Decode



# Disaggregating Prefill and Decode



# Terminology and Metrics

**Time per Output Token (TPOT):** Average time for subsequent token generation (**Decode**)

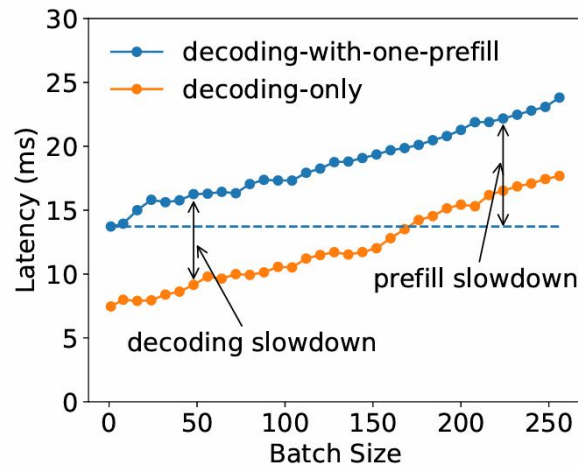
**Instance:** a unit of resources that manages exactly one complete copy of model weights

**Goodput:** # of completed requests *that meet the SLO* per second

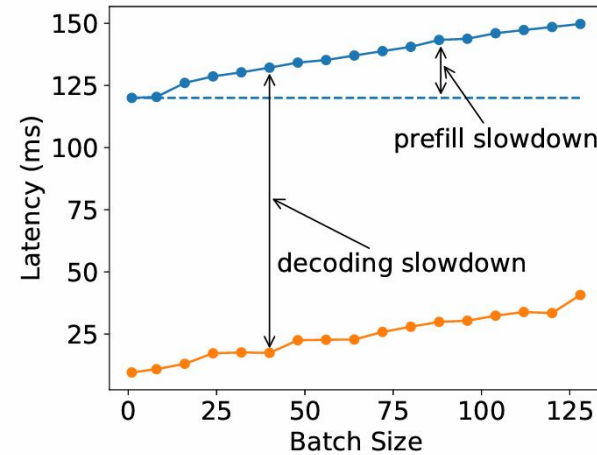
**Disaggregation:** Treating the prefill and the decode as two separate “programs” operating on different instances

# Disaggregation Motivation

As you increase the prompt length, the time that gets wasted for the decode in a chunked prefill/decode mechanism becomes very large



(a) Input length = 128

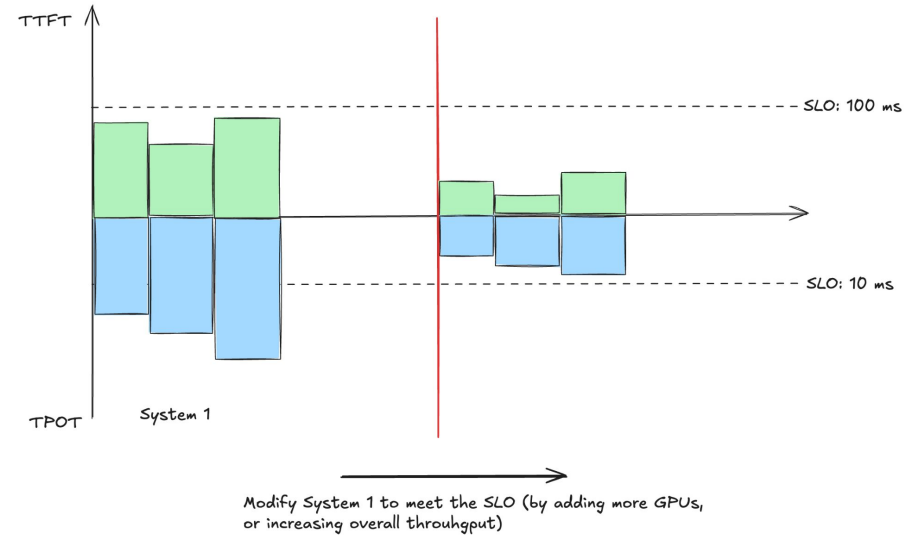


(b) Input length = 1024



# Disaggregation Motivation

- If we take the naive method to meet the SLO, we reduce overall prefill and decode latency
- **However:** with disaggregation, we get fine-grained control, and can modulate the TPOT/TTFT to meet the SLO independently
- Prioritization of TPOT/TTFT based on the use case
- Disaggregation -> Better Goodput

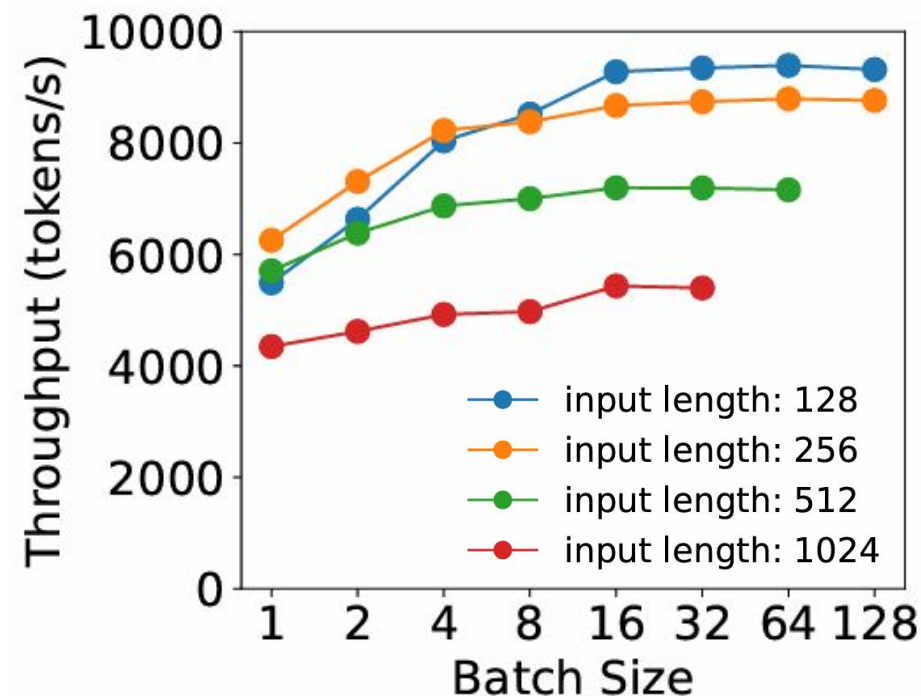


# Analysis for Prefill Instance

## Compute-bound

The paper denotes  $L_m$  as a **critical input length threshold** beyond which the prefill phase becomes compute bound

What do you think is the estimate  $L_m$  in the following figure?



(a) Prefill phase

# M/D/1 queue

The formula for the average waiting time in an M/D/1 queue

$$W = \frac{\rho \times d}{2(1 - \rho)}$$

$d$  is the deterministic service time

$\rho$  (utilization) is the ratio of arrival rate to service rate

M (Markovian / Poisson Arrivals) describes how requests arrive at the system.

D (Deterministic Service Time) describes how long it takes to process a request.

1 (Single Server) represents the number of processing units.

First-Come-First-Served (FCFS)

# Parallelism Summary

Assume **uniform requests input**  
**lengths of 512 tokens** and a Poisson  
arrival process

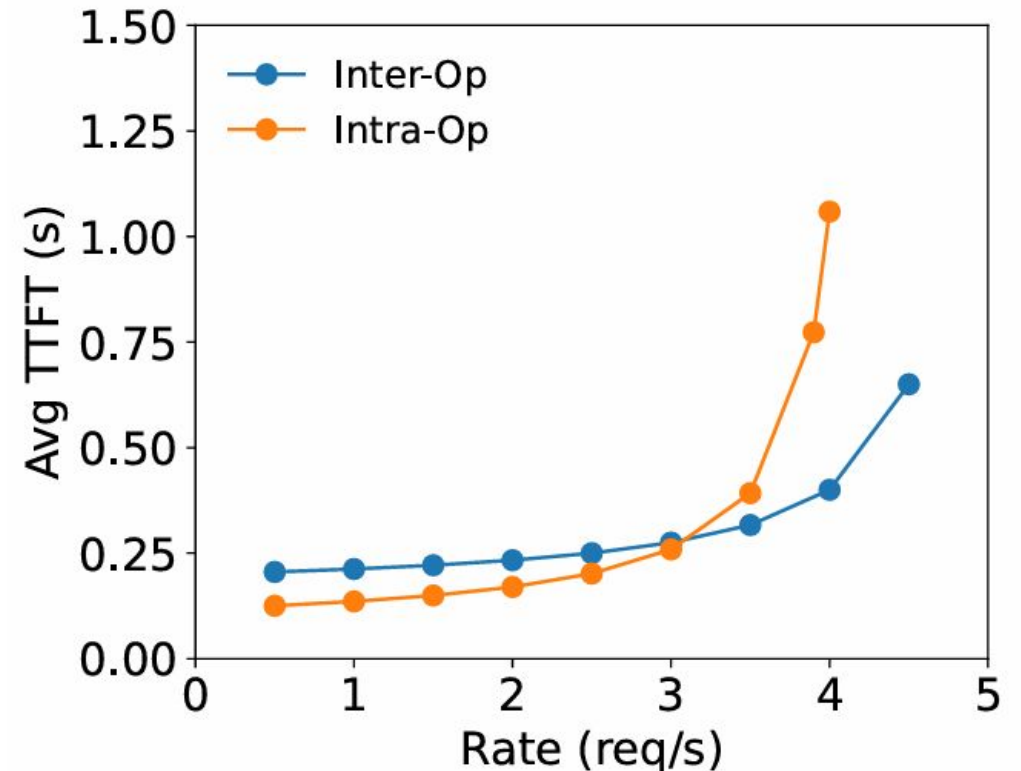
At lower rates, where execution time is  
the primary factor, intra-op parallelism  
is more efficient

As the rate increases, inter-op  
parallelism becomes advantageous

$$Avg\_TTFT = D + \frac{RD^2}{2(1 - RD)}.$$

$$Avg\_TTFT_{intra} = \frac{D}{K} + \frac{RD^2}{2K(K - RD)}.$$

$$Avg\_TTFT_{inter} = D_s + \frac{RD_m^2}{2(1 - RD_m)} = D + \frac{RD^2}{4(2 - RD)}.$$





# Parallelism plan: inter-op

The prefill phase functions analogously to an M/D/1 queue

$$Avg\_TTFT = D + \frac{RD^2}{2(1 - RD)}.$$

D - execution time for each request

R - the Poisson arrival rate

Assume utilization condition of  $RD < 1$

With 2-way inter-op parallelism, the paper assume the request-level latency becomes  $D_s$ , and the slowest stage takes  $D_m$  to finish

$$D \approx D_s \approx 2 \times D_m$$

$$Avg\_TTFT_{inter} = D_s + \frac{RD_m^2}{2(1 - RD_m)} = D + \frac{RD^2}{4(2 - RD)}.$$

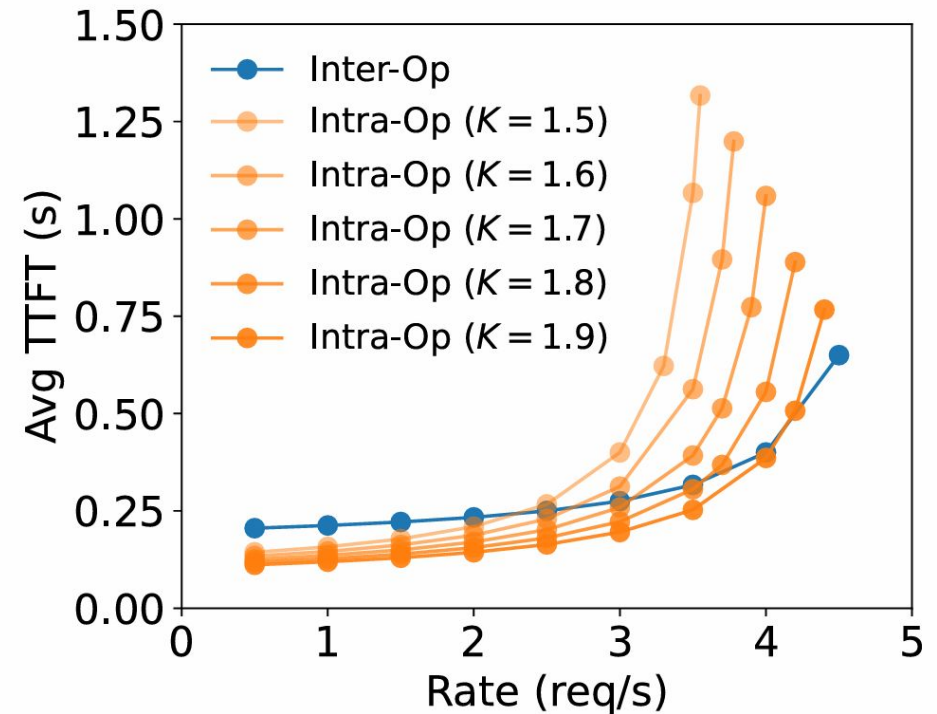
# Parallelism plan: intra-op

For intra-op parallelism, the paper introduces a speedup coefficient  $K$ , where  $1 < K < 2$ , reflecting the imperfect speedup caused by high communication overheads

Execution time -  $D_s = D / K$

$$Avg\_TTFT_{intra} = \frac{D}{K} + \frac{RD^2}{2K(K - RD)}.$$

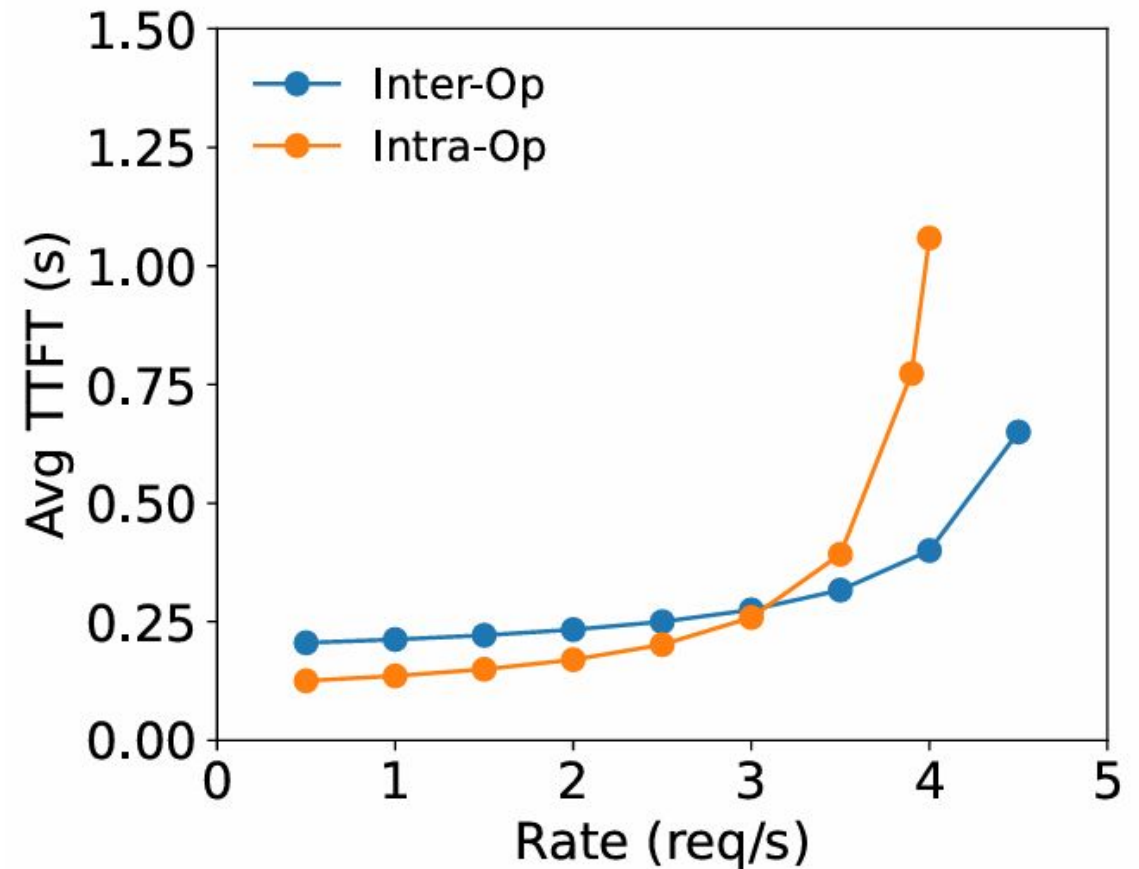
The value of  $K$  depends on factors such as the input length, model architecture, communication bandwidth, and placement



# Parallelism Summary

At lower rates, where execution time is the primary factor, intra-op parallelism is more efficient.

As the rate increases, inter-op parallelism becomes advantageous

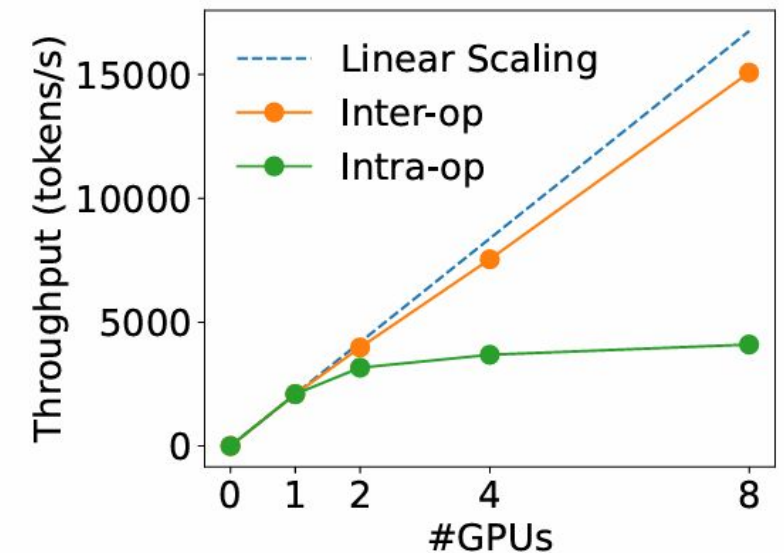
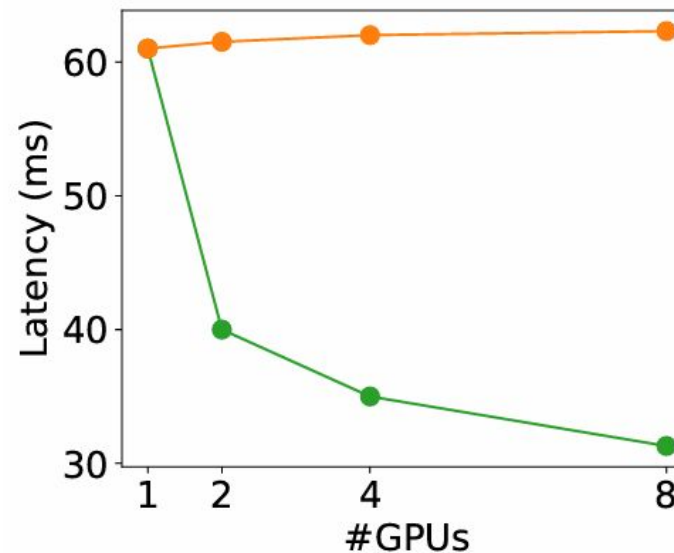


# Analysis for Decoding Instance

## Memory-bound

Intra-op parallelism reduces latency with diminishing returns, caused by communication and reduced utilization after partition

Inter-op parallelism can almost linearly scale at throughput





# Considerations

Smaller models that can fit GPUs

Variable prefill length

Communication overhead

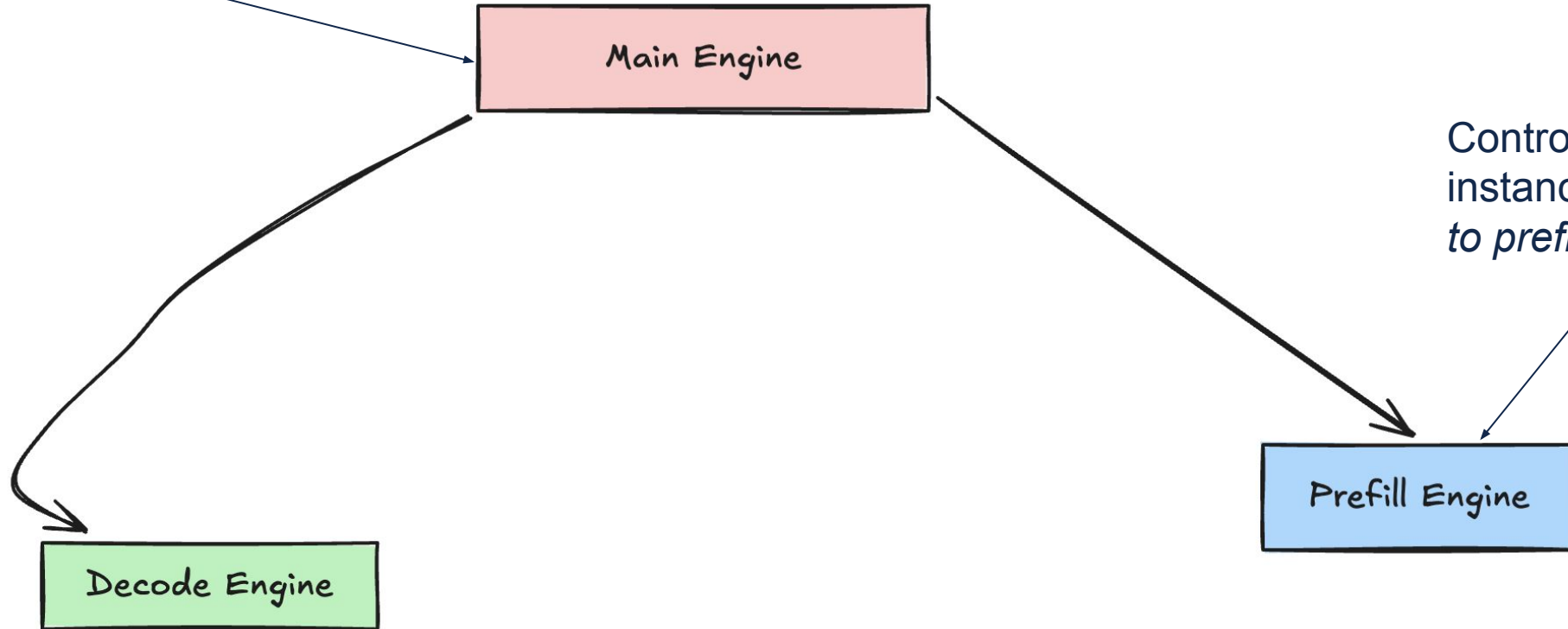
$$Avg\_TTFT = D + \frac{RD^2}{2(N - RD)}$$

DistServe implementation addresses  
these problems in more detail

# Implementation

# Implementation - DistServe Architecture

Spawns prefill and decode engines



Controls/spawns GPU instances *solely dedicated to prefill*

Controls/spawns GPU instances *solely dedicated to decoding*

# Implementation - DistServe Architecture

Spawns prefill and decode engines

Main Engine

Controls/spawns GPU instances *solely dedicated to prefill*

Prefill Engine

Decode Engine

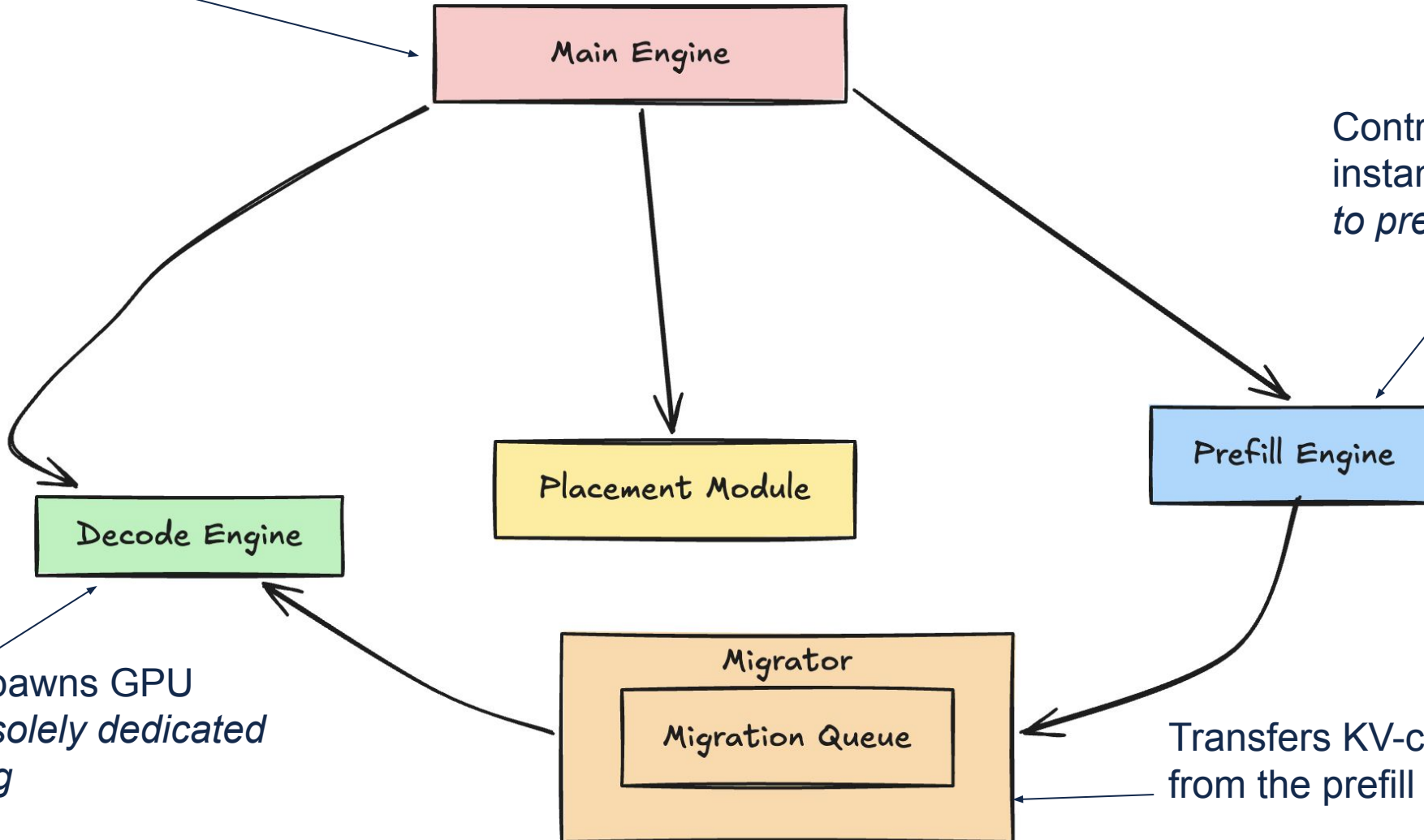
Controls/spawns GPU instances *solely dedicated to decoding*

Placement Module

Migrator

Migration Queue

Transfers KV-caches from the prefill to decode



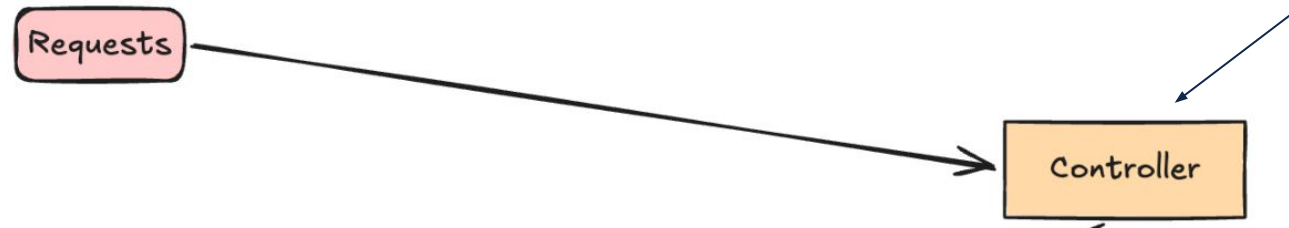


# Key Considerations

**How does the allocation of resources change when the communication bandwidth is lower/higher? (for example NVLink vs PCIe)**

- Allocate instances in such a way that KV caches are *a/ways* transferred over high-bandwidth interconnect

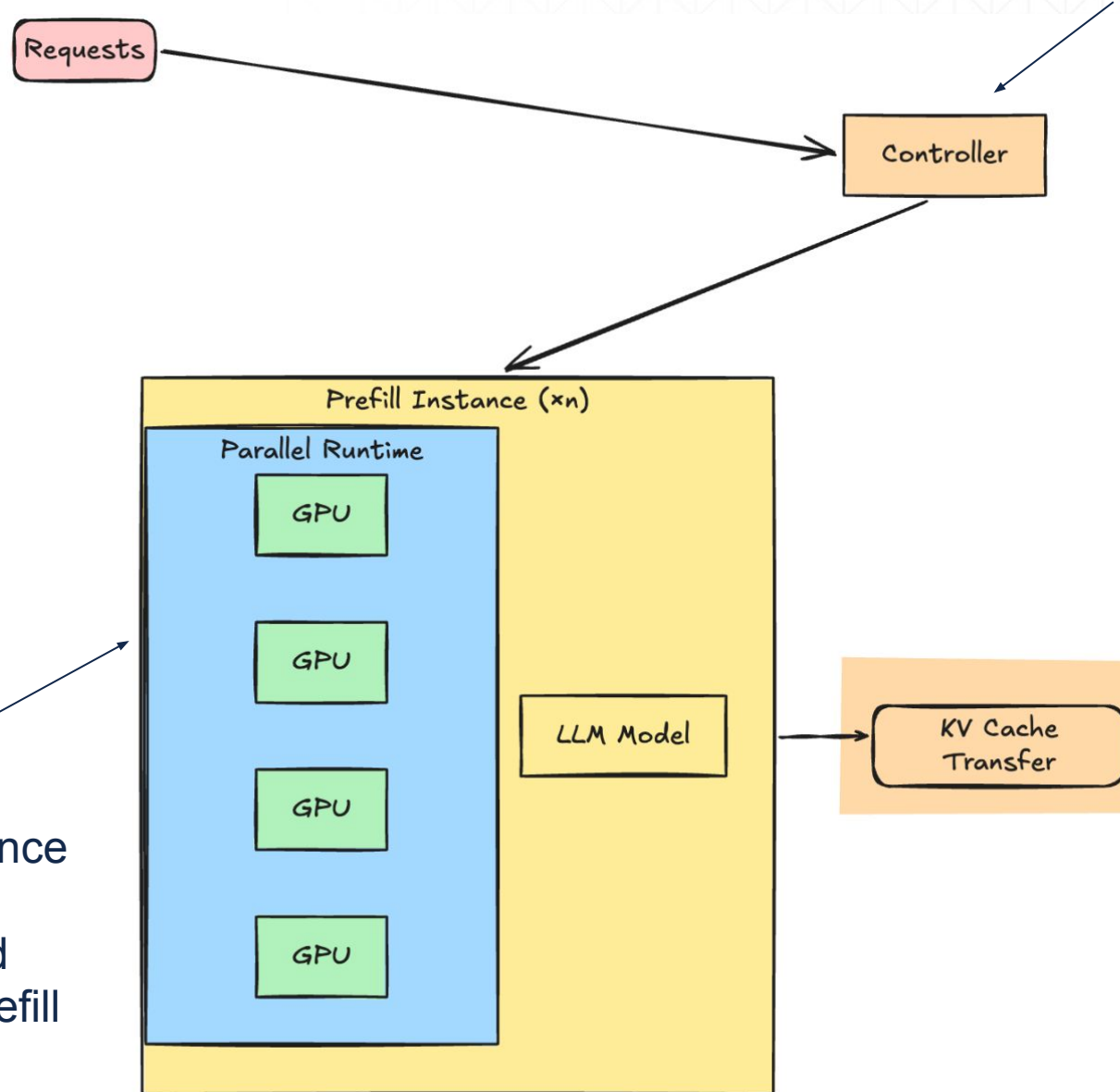
# Main Engine (Scheduler)



Controller responsible for scheduling requests

# Prefill Engine

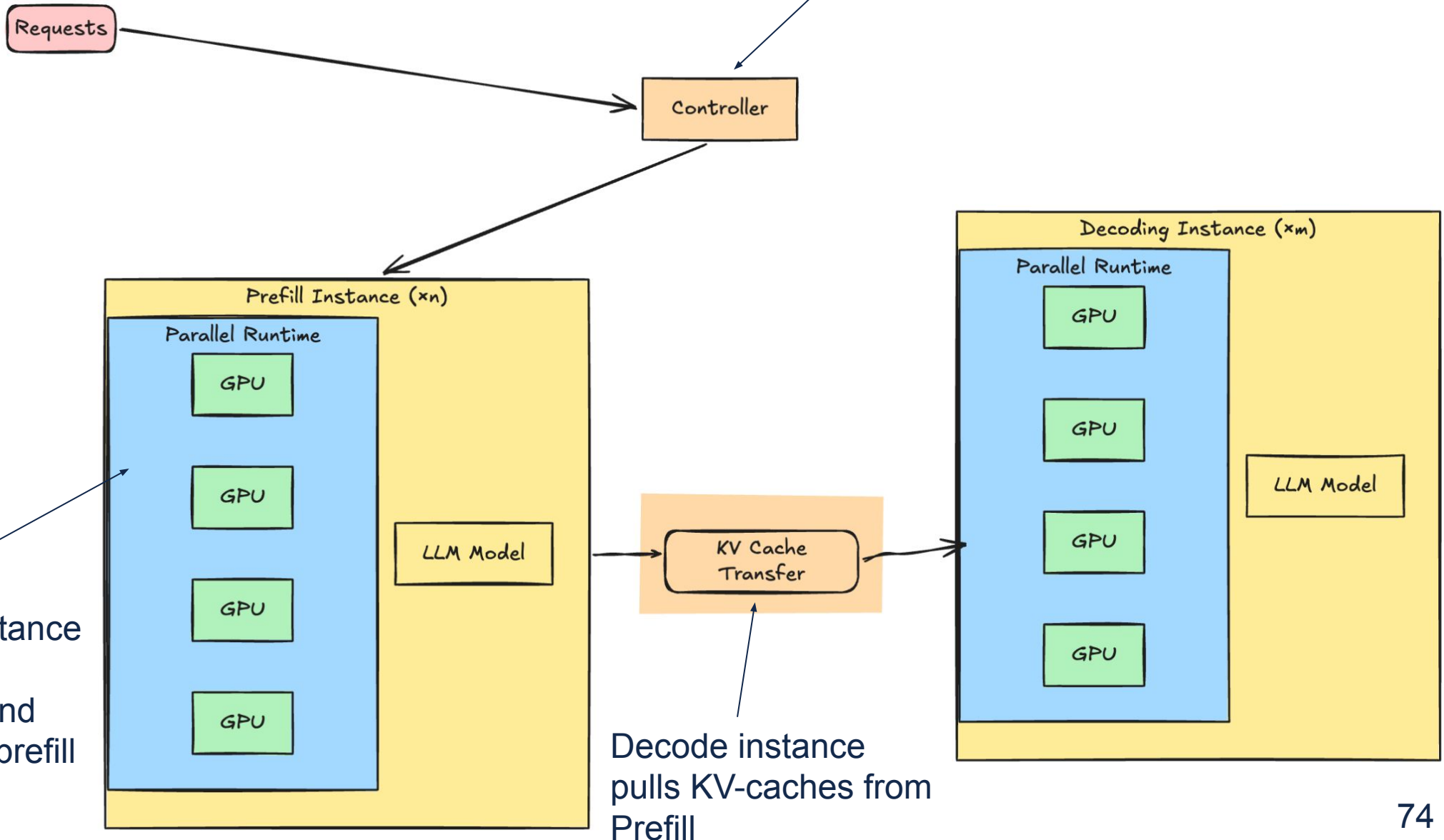
Uses a First-Come-First-Serve dispatch policy, picks the prefill instance with the smallest queue



Dedicated prefill instance gets configured by placement engine and based on historical prefill use-traces

# Decode Engine

Uses a First-Come-First-Serve dispatch policy, picks the prefill instance with the smallest queue





**What can be done about pipeline  
bubbles?**

# Mitigating Pipeline Bubbles

**Prefill:** profile target model to find minimal prompt length to saturate the GPU. Schedule batches with a total sequence length close to this length. Prompts larger than  $L_m$  tokens get scheduled individually, while you batch together many smaller prompts to form a  $L_m$ -sized batch

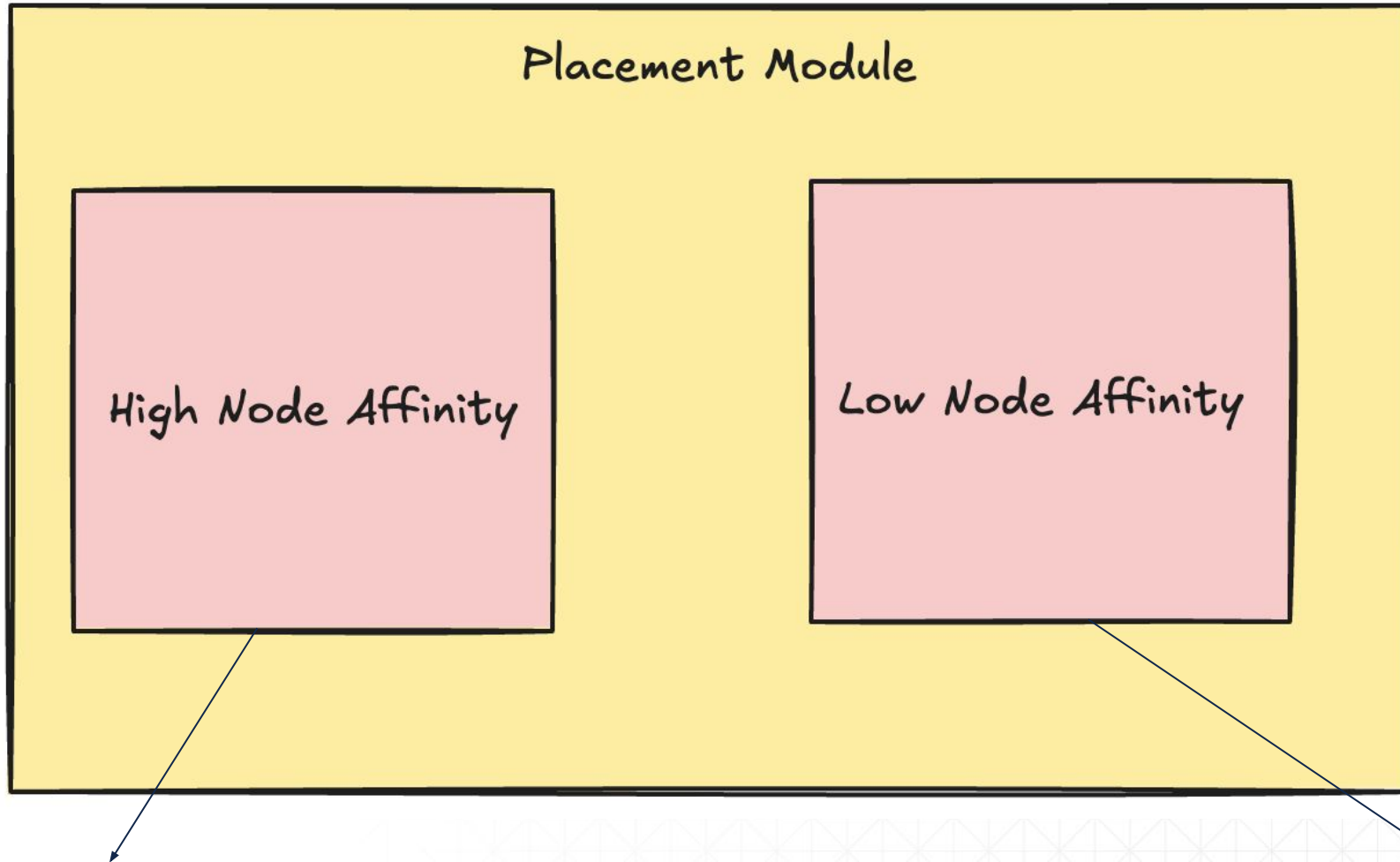


# Mitigating Pipeline Bubbles

**Decode:** The decoding instance maintains a batch size near  $L_m$  by continuously pulling completed KV caches from whichever prefill instances have finished processing requests, replacing slots freed by completed generations



# Placement Module



Decides how to place each instance on the GPU cluster

High Node Affinity

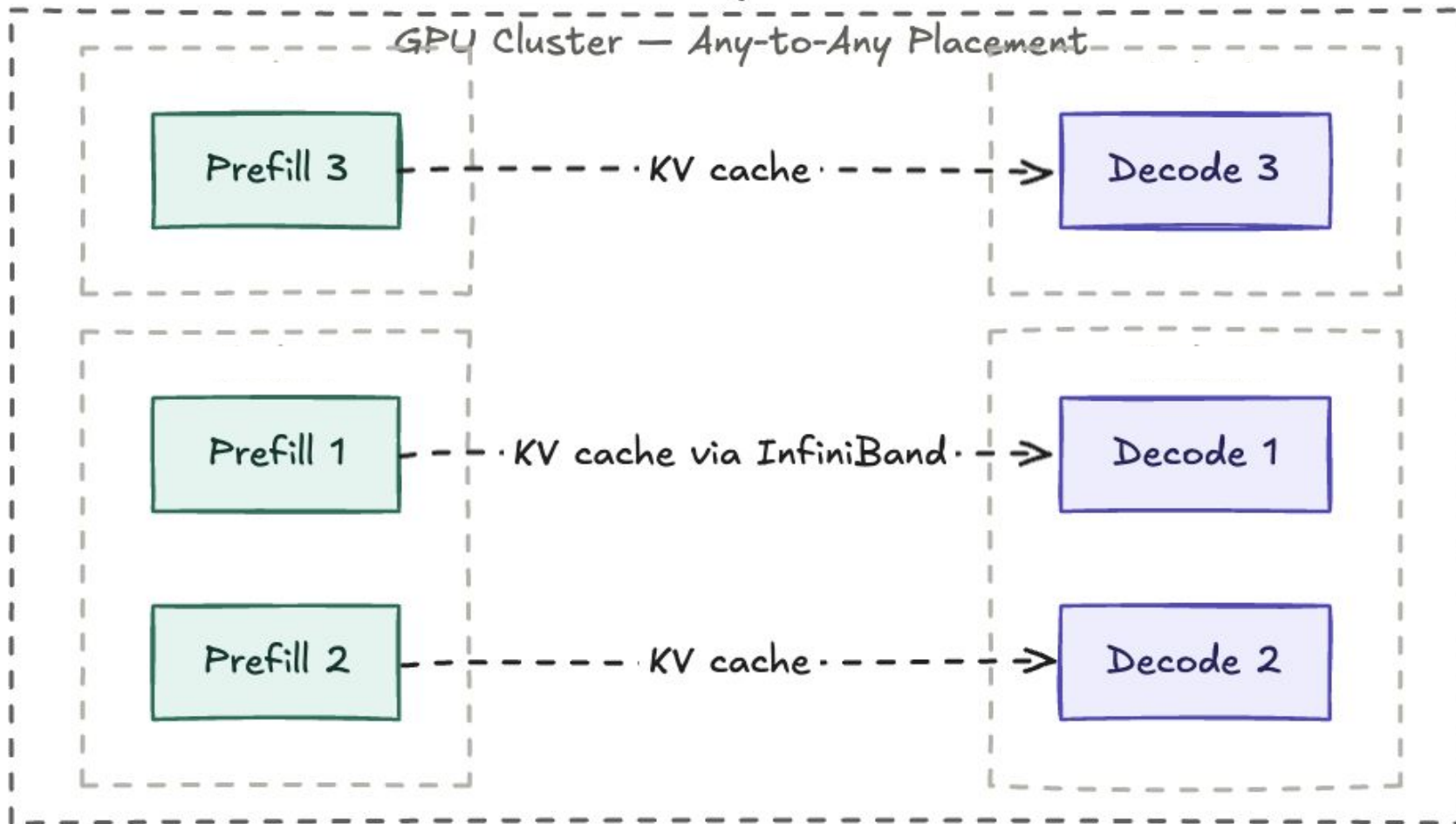
Low Node Affinity

Can place Prefill/Decode instances on different clusters, so node placement is a non-constraint - KV Cache transfer is negligible

**To ensure KV cache transfer isn't a huge bottleneck:** prefill & decode must be colocated on the same node

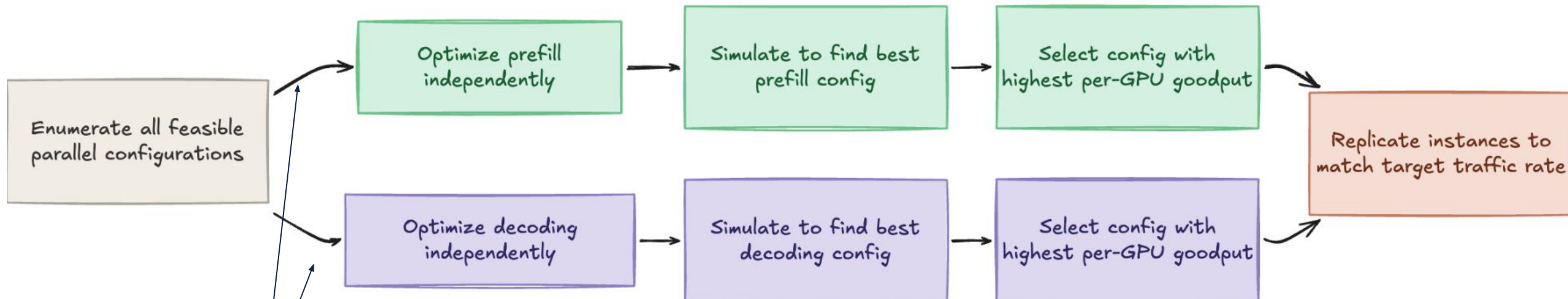


# High Node Affinity



Find optimal configuration for a single prefill instance and a single decode instance, and **match the overall traffic rate by replicating the instances**

# High Node Affinity



Degrees are valid on one condition:

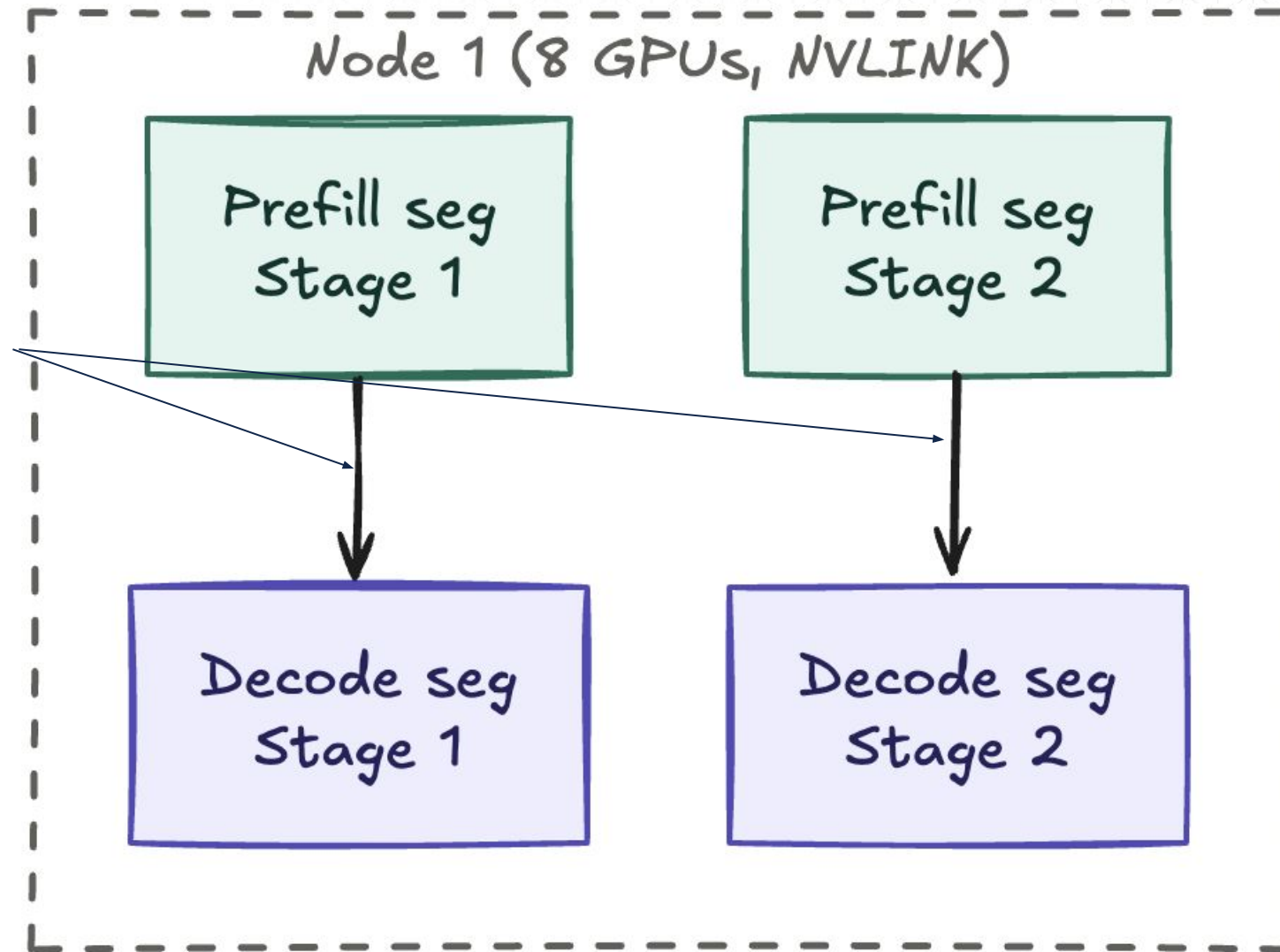
$$\text{LLM size} / (\text{inter\_op} * \text{intra\_op}) < \text{Memory Capacity}$$

where `inter_op` is the number of pipeline stages, and  
`intra_op` is the number of GPUs per pipeline stage

**How can we limit the bottleneck of  
transferring KV caches for Low  
Affinity Cases?**

# Low Node Affinity

We only really need to transfer the KV-caches for **corresponding layers of the prefill and decode stages**

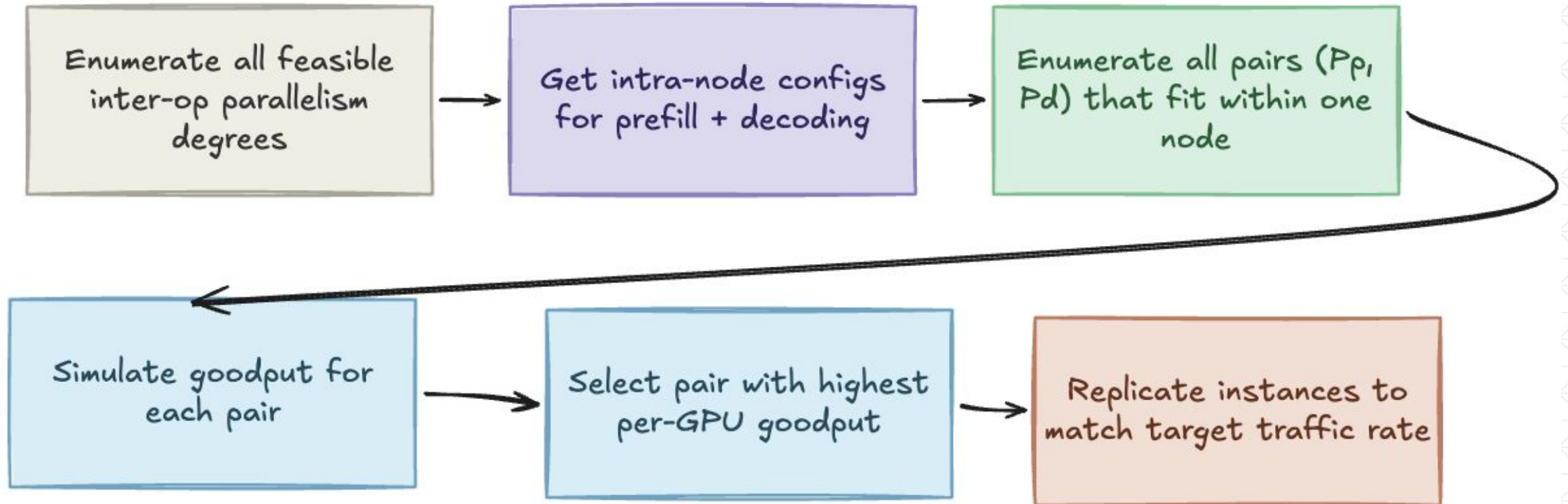


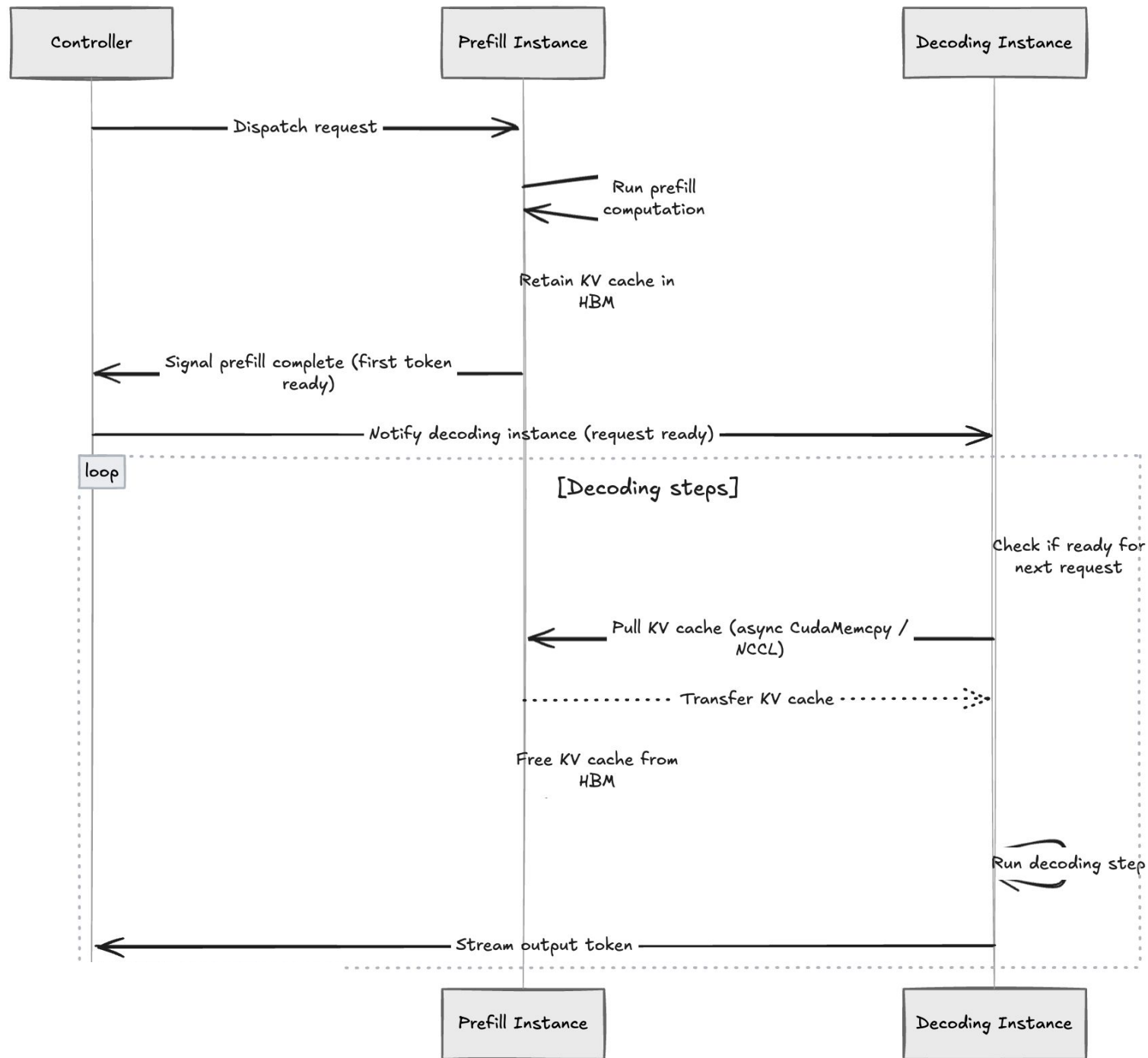
KV Cache transfers are done intra-node



# Low Node Affinity

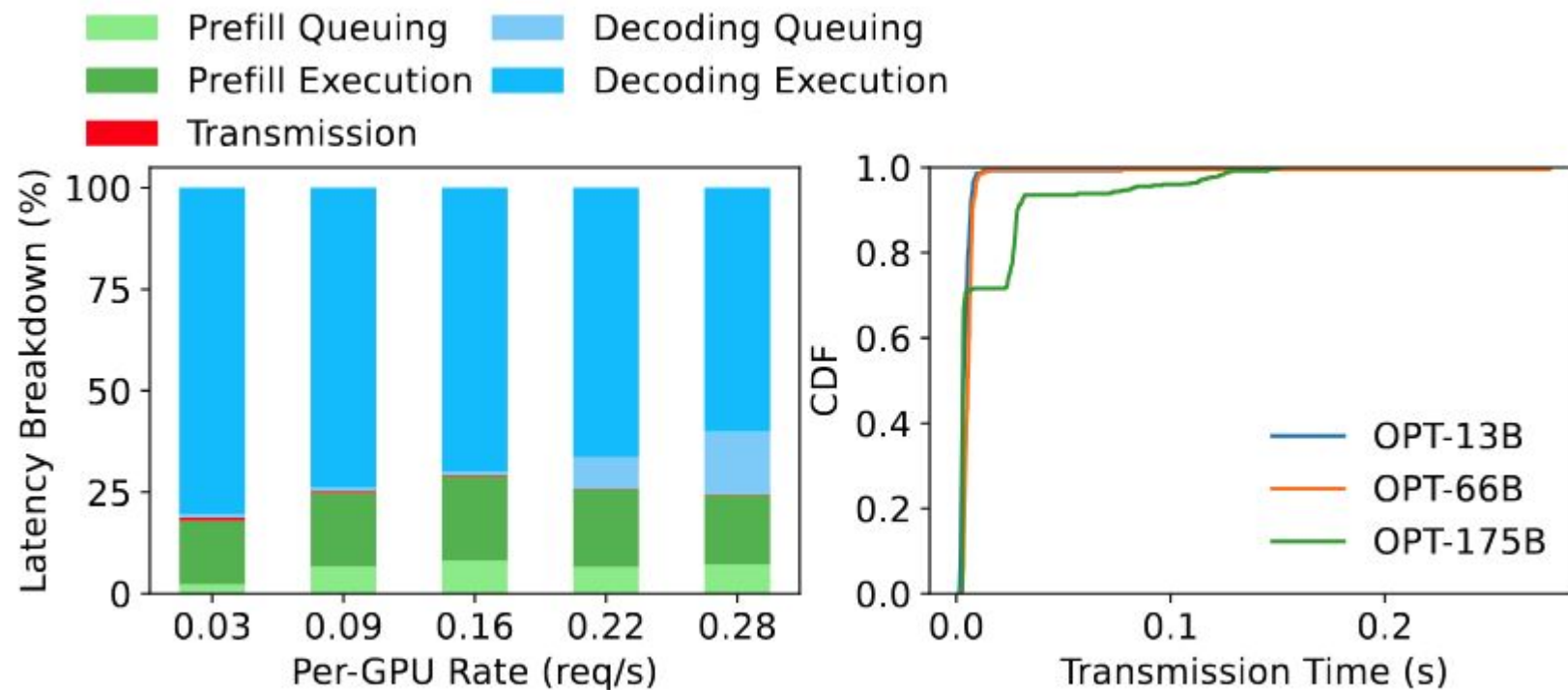
The pipeline stage partitioning across nodes is same for prefill and decode - But how the stage gets mapped to a node is different (can have different intra-node parallelism degrees)





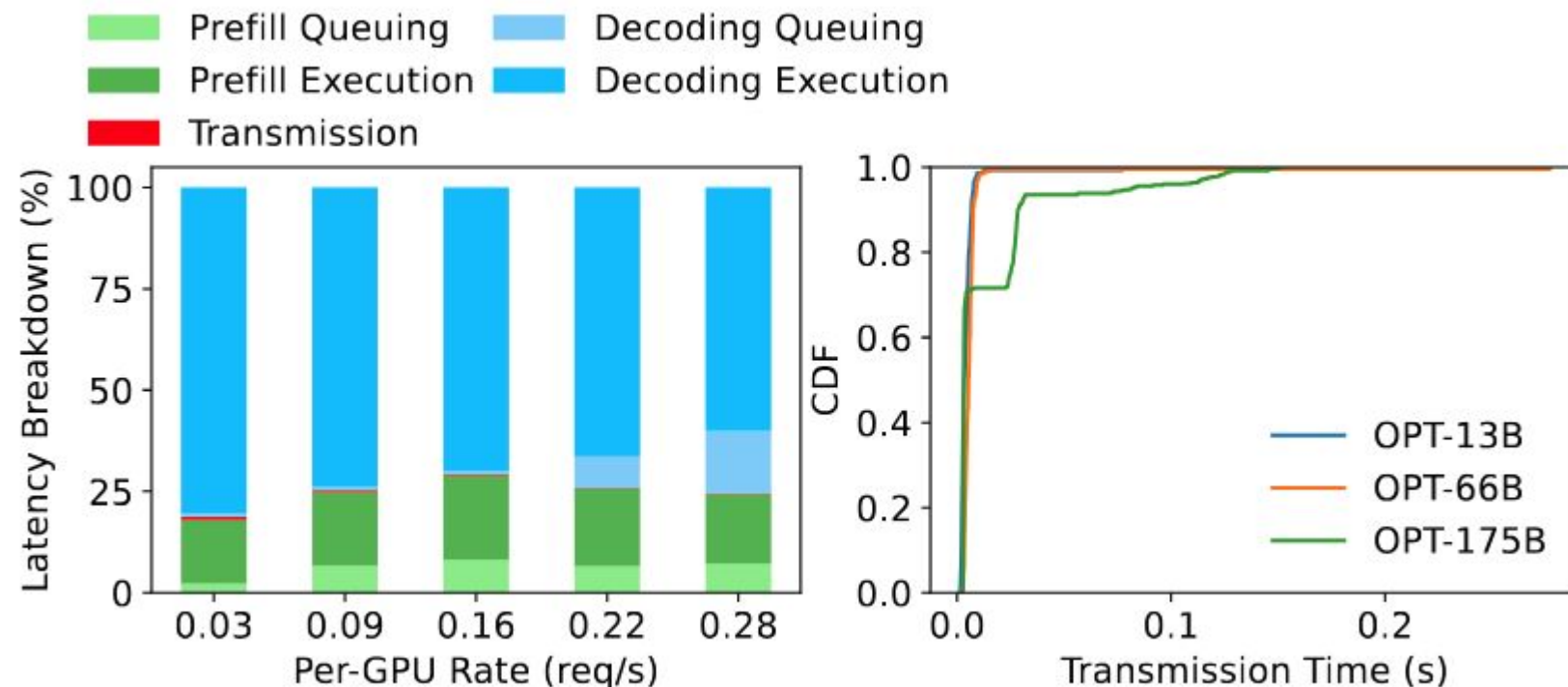
# Latency Breakdown

- Running OPT 175B on a low-affinity cluster



# Latency Breakdown

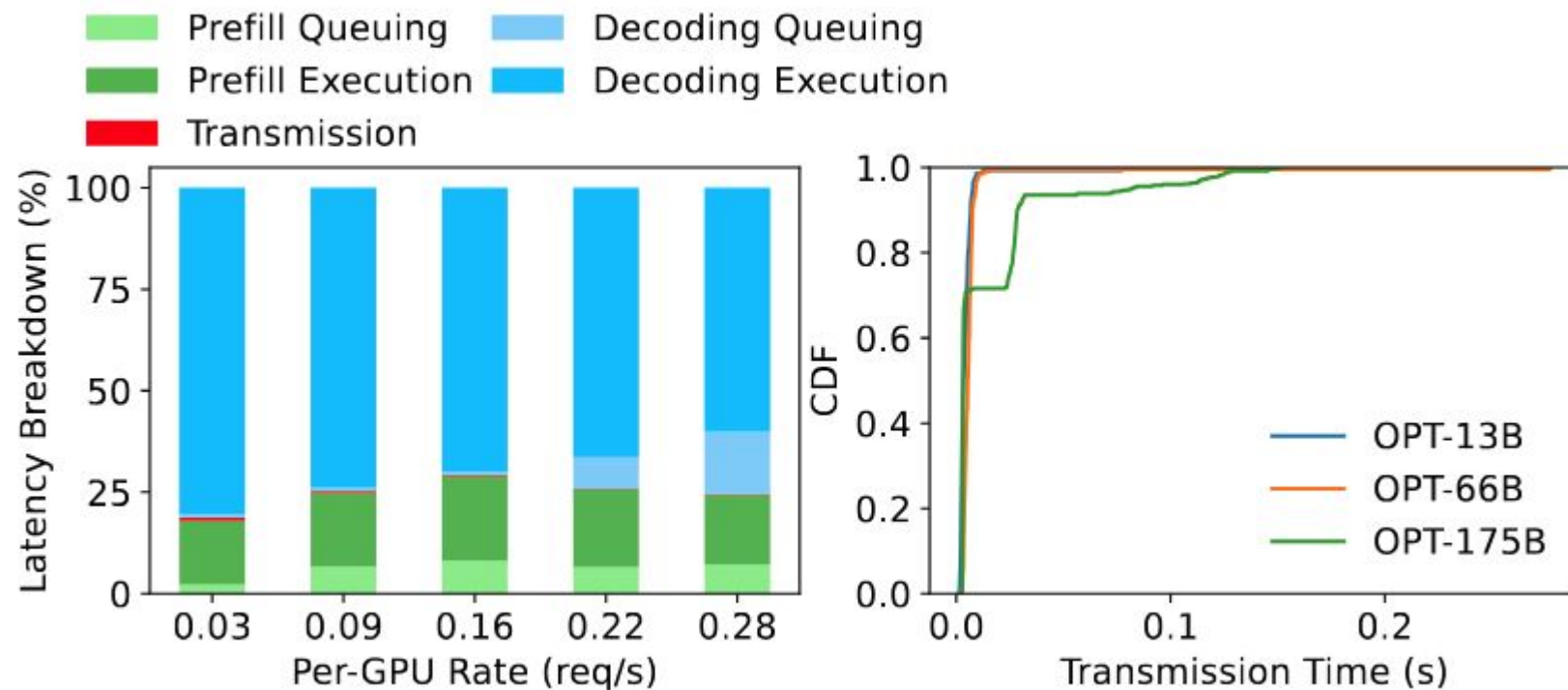
- Running OPT 175B on a low-affinity cluster
- Time spent in KV-Cache transfer phase is very low despite low-affinity cluster!





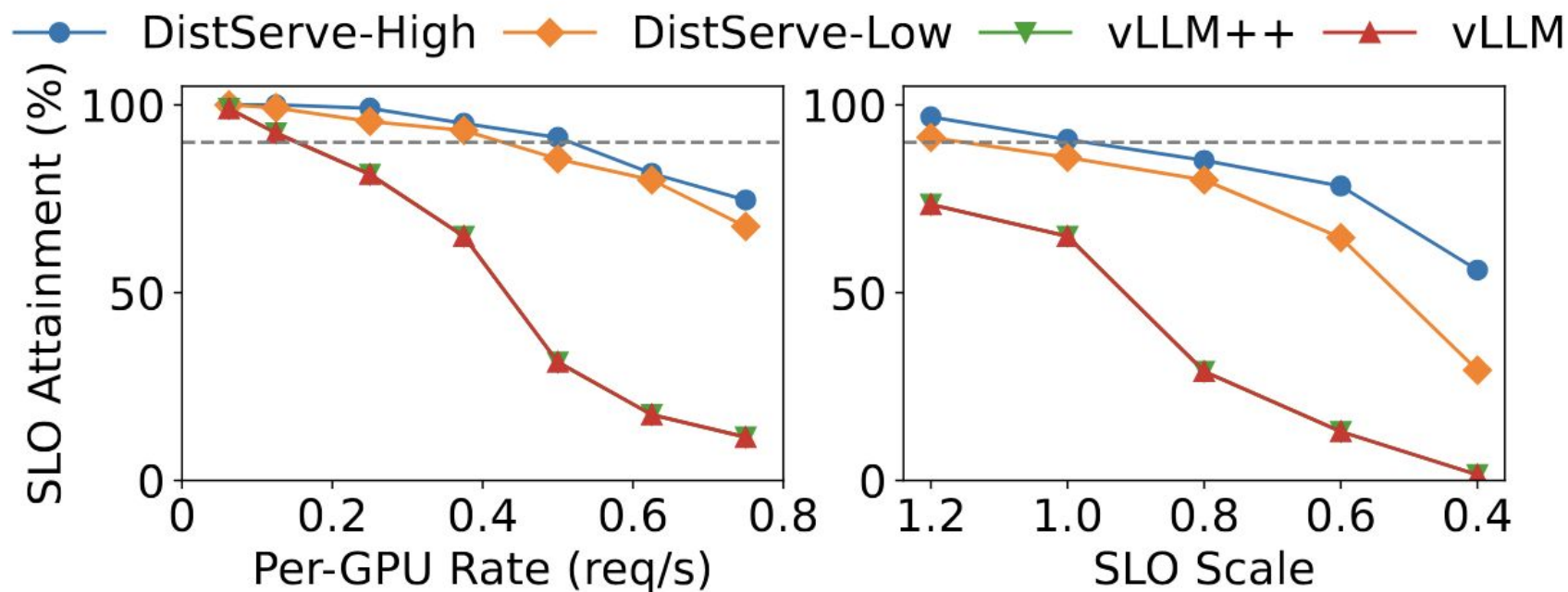
# Latency Breakdown

- Running OPT 175B on a low-affinity cluster
- Time spent in KV-Cache transfer phase is very low despite low-affinity cluster!
- 95% of requests experience a delay of less than 30ms



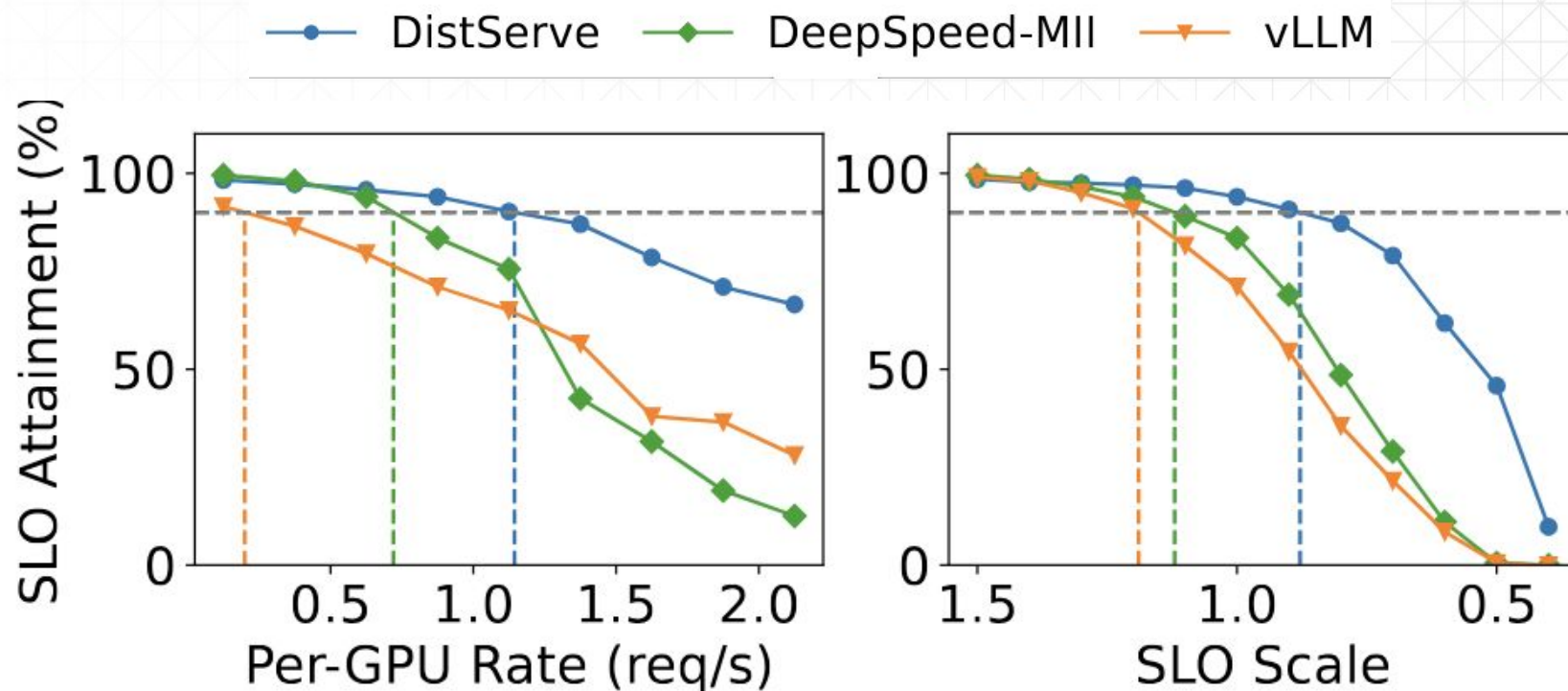
# Ablation Experiments

- vLLM++ is an extension of vLLM that implements the pipeline strategy search described in this paper
- Disaggregation is extremely important for high SLO attainment
- High Bandwidth interconnect allows for more parallelism strategies to be enumerated



# Overall Results - Code Completion

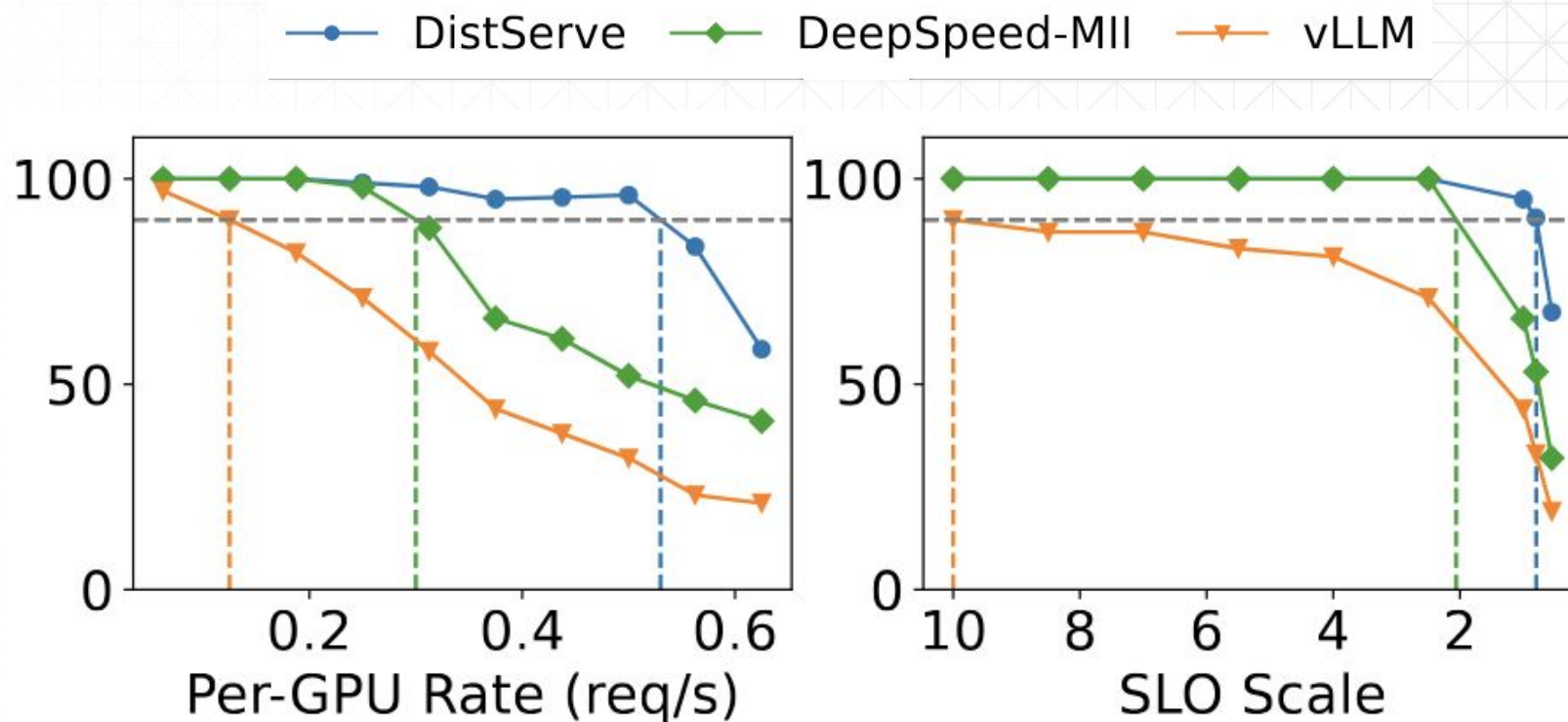
- Real-time code completion agent requires very low TTFT
- DistServe meets the TTFT requirements by reducing average latency of prefill jobs through eliminating the interference of decoding jobs
  - Also by increasing the intra-op parallelism inside a prefill instance



(a) Code Completion

# Overall Results - Summarization

- TTFT is not a requirement, but benefits from very low TPOT. Summarization tasks have very long prefill inputs.
- Since vLLM does collocation, it experiences high latency in the decode step due to the latency of one prefill
- Disaggregation helps a lot for this task due to the sheer difference between the computation requirements of both phases

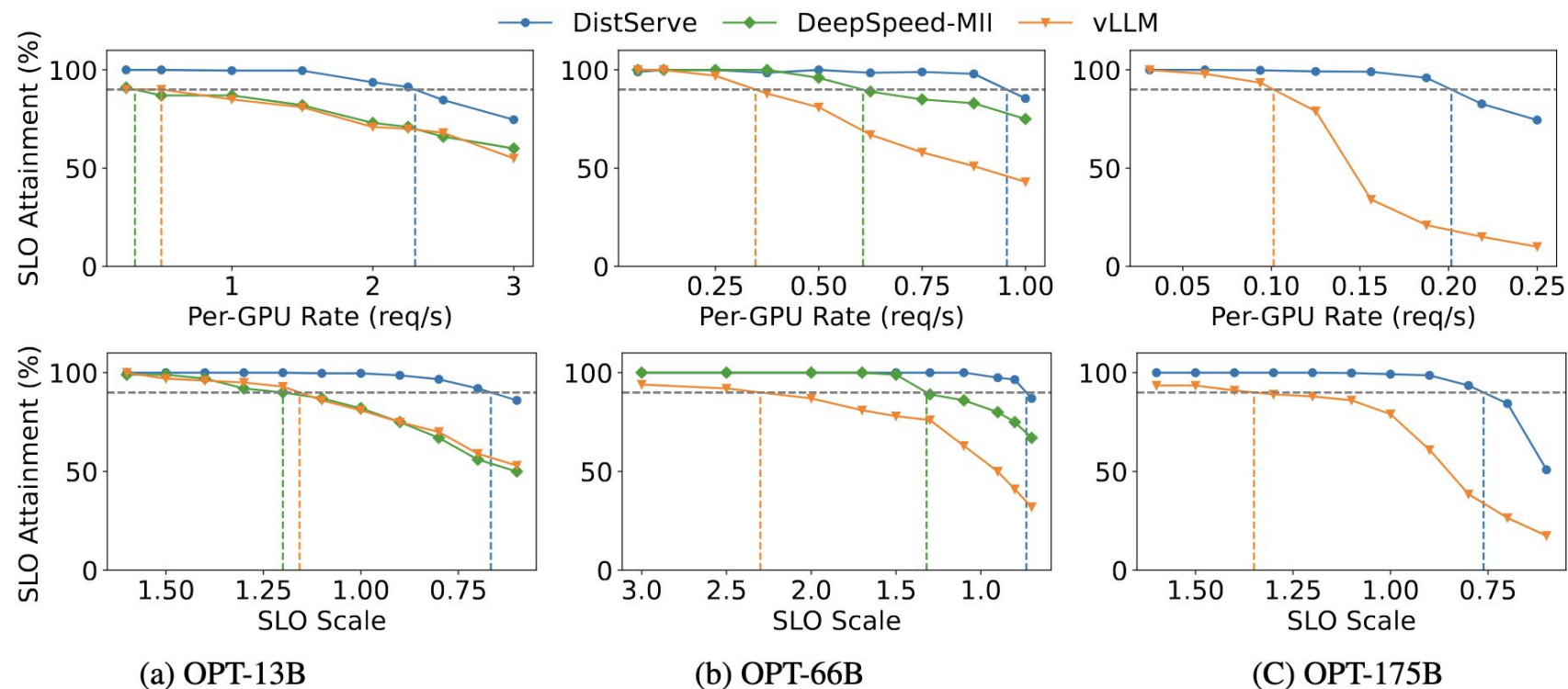


(b) Summarization



# Overall Results - Chatbot

- Using an Chatbot's SLO, they found that the disaggregated strategy consistently beats other techniques for both low and high per-GPU request rates
- Even as they vary and target stricter SLO requirements (SLO scale), the disaggregated strategy consistently beats traditional collocation and priority based techniques



# Future Work

## DistServe

- Evaluate fault propagation between prefill and decode instances (if there is a error in the prefill, it will propagate to the decode)
- Preemption: interrupt a currently running request to let a higher priority one go first (mitigate convoy effect)

## Sarathi-Serve

- The system performance can be further enhanced by dynamically varying the token budget based on workload characteristics.



**Thank you**

Chau, Seokjoo, Minseob, Neil